

Foundations of Spatial AI

Lecture 03 — Single-View Depth

Detailed lecture notes

Notes based on the slides of L. Di Giammarino and E. Giacomini
(Sapienza University of Rome, 30 September 2026)

How to read these notes. Section headers reuse the titles of the slides and are tagged with the slide range they cover. Formal statements (definitions, propositions, theorems, algorithms) are placed where the slides introduce the topic; the slides themselves state most results without proof, so the derivations are added here. Blue *Intuition* boxes explain the “why” in plain words, with worked numbers. All figures are original TikZ/pgfplots drawings; figures marked *schematic* show the qualitative trend described on the slides, not measured data.

Contents

3.1 From Image Formation to Measurement	(slides 1–3)	4
3.2 From Representations to Measurements	(slides 4–7)	5
3.2.1	Recovered and Ambiguous Quantities (slides 5)	5
3.2.2	The Back-Projected Ray (slides 6)	5
3.2.3	Scale-Depth Ambiguity (slides 7)	6
3.3 Three Ways to Pay for Depth	(slides 8–12)	7
3.3.1	Two Measurement Principles (slides 9)	7
3.3.2	Triangulation From Two Views (slides 10)	8
3.3.3	The Three Currencies (slides 11)	9
3.3.4	The Single-Image Claim (slides 12)	9
3.4 Inside a LiDAR	(slides 13–17)	9
3.4.1	Pulsed Time of Flight (slides 14)	9
3.4.2	Structured Sparsity (slides 15)	10
3.4.3	Limitations of a LiDAR Point Cloud (slides 16)	11
3.4.4	Two Error Laws (slides 17)	11
3.5 From Point Cloud to Depth Image	(slides 18–19)	12
3.5.1	Projecting a Cloud Into the Image (slides 19)	12
3.6 From Recovery to Prediction	(slides 20–22)	13
3.6.1	Monocular Cues in Human Vision (slides 21)	13
3.6.2	Depth Prediction as a Prior (slides 22)	14

3.7 Cues in Closed Form (slides 23–26)	14
3.7.1 Vanishing Points (slides 24)	14
3.7.2 Apparent Size (slides 25)	16
3.7.3 Vertical Position (slides 26)	16
3.8 Supervised Single-View Depth (slides 27–35)	17
3.8.1 The Supervised Setup (slides 28)	17
3.8.2 Three Datasets (slides 29)	18
3.8.3 The Scale-Invariant Loss (slides 30)	18
3.8.4 Output Spaces (slides 31)	19
3.8.5 Affine Invariance in Disparity and in Depth (slides 32)	20
3.8.6 MiDaS: Mixing the Datasets (slides 33)	21
3.8.7 Depth Anything V2 (slides 34)	22
3.8.8 Failure on Anamorphic Art (slides 35)	22
3.9 Cues Used by the Network (slides 36–39)	23
3.9.1 The Pasted-Car Experiment (slides 37)	23
3.9.2 Pitch and Roll (slides 38)	23
3.9.3 Dependence on the Training Camera (slides 39)	24
Next: Self-Supervision and a Generative Prior (slide 40)	25
Overview: Part I in One Slide (slides 1–2)	25
3.10 Supervision without Ground Truth (slides 3–6)	26
3.10.1 View Synthesis and the Backward Warp (slides 4)	26
3.10.2 Image Reconstruction as Supervision (slides 5)	27
3.10.3 Where the Loss Is Blind (slides 6)	28
3.11 Diffusion, From Scratch (slides 7–22)	29
3.11.1 Supervised Learning (slides 8)	29
3.11.2 Generative Learning (slides 9)	29
3.11.3 Many Outputs for One Input – Text to Image (slides 10)	29
3.11.4 The Forward and Reverse Process (slides 11)	30
3.11.5 Training by Noise Prediction (slides 12)	31
3.11.6 The Spiral Toy Model (slides 13)	31
3.11.7 Training the Toy Model (slides 14)	32
3.11.8 The Ideal Denoiser (slides 15)	33
3.11.9 The Smoothed Distance Function (slides 16)	33
3.11.10 Sampling From a Diffusion Model – Why iterative? (slides 17)	34
3.11.11 The DDIM Sampler (slides 18)	35
3.11.12 Probabilistic Sampling (slides 19)	35
3.11.13 Three Uses of a Prior (slides 20)	36
3.11.14 The Autoencoder and Latent Diffusion (slides 21)	36
3.11.15 Conditioning and Guidance (slides 22)	37
3.12 Marigold (slides 23–31)	37
3.12.1 The Marigold Method (slides 24)	37
3.12.2 From the Spiral to Marigold (slides 25)	38
3.12.3 The Source of Affine Invariance (slides 26)	38
3.12.4 Conditioning on the Image (slides 27)	39
3.12.5 Training Marigold (slides 28)	39
3.12.6 Inference (slides 29)	40

3.12.7	Ensembling Over Samples (<i>slides 30</i>)	41
3.12.8	Limitations of Marigold (<i>slides 31</i>)	41
3.13	Back to 3D (<i>slides 32–34</i>)	41
3.13.1	Effect of Scale and Shift Errors (<i>slides 33</i>)	41
3.13.2	Sources of Scale and Shift (<i>slides 34</i>)	42
3.14	Marigold-DC (<i>slides 35–38</i>)	42
3.14.1	The Depth Completion Task (<i>slides 36</i>)	43
3.14.2	Steering the Loop (<i>slides 37</i>)	44
3.14.3	Nothing Retrained (<i>slides 38</i>)	44
	Summary (<i>slides 39–40</i>)	46
A	Supplementary derivations (not on the slides)	48

3.1 From Image Formation to Measurement (*slides 1–3*)

Lecture 01 modeled how a scene becomes an image: projection sends a 3D point to a pixel and *discards the distance along the ray*. Lecture 02 covered ways to *represent* a scene: points, meshes, voxels, signed distance functions, Gaussians. Neither lecture covered how to *measure* a scene. This lecture obtains one depth value per pixel (not always – some sensors leave holes) in one of three ways: by measuring it with a laser, by stereo vision, or by *predicting* it with a learned prior.



Section	Title	Slides
3.1	From representations to measurements	4–7
3.2	Three ways to pay for depth	8–12
3.3	Inside a LiDAR	13–17
3.4	From point cloud to depth image	18–19
3.5	From recovery to prediction	20–22
3.6	Cues in closed form	23–26
3.7	Supervised single-view depth	27–35
3.8	Cues used by the network	36–39

Table 1: The agenda (slide 3) with slide ranges.

Intuition

The whole lecture is one story: an image is a *projection*, so depth is missing. Every method for getting it back must *pay* with something – time (a laser pulse), a baseline (a second viewpoint), computation (motion), or a prior (assumptions learned from data). The interesting questions are always: what did we pay, and what is therefore *guaranteed* about the answer?

3.2 From Representations to Measurements (*slides 4–7*)

What one image determines, and what it does not. The two preceding lectures modeled image formation and scene representation. This section separates *recovered* from *ambiguous* quantities and shows which of the two depth from one image is.

3.2.1 Recovered and Ambiguous Quantities (*slides 5*)

Definition 3.2.1 (Recovered vs. ambiguous quantity). Every quantity estimated in this lecture falls into one of two cases.

- **Recovered:** the data determine it up to noise. It can be reported in meters with an error bar.
- **Ambiguous:** the data determine it only up to a *family of transformations*. Any single number reported for it comes from an added assumption.

Quantity	Free parameters	What pins it down
metric depth	0	nothing, it is already in meters
up to scale	1 (s)	one known length
up to scale and shift	2 (s, t)	two known depths
ordering only	any monotone map	a calibrated sensor

Table 2: Ambiguity ladder (slide 5). The number of free parameters says how much external information is still needed.

Intuition

Think of the free parameters as *knobs you cannot read from the data*. Metric depth has no knob. “Up to scale” has one knob: the scene may be uniformly inflated or shrunk. “Up to scale and shift” has two. “Ordering only” is the weakest: you know that A is nearer than B , but $(1, 2, 3)$ and $(1, 10, 100)$ tell exactly the same story. The slide’s punchline: disagreements about depth estimation are usually disagreements about *which row of this table a quantity lives in*.

3.2.2 The Back-Projected Ray (*slides 6*)

Projection throws one number away per pixel; inverting it asks for that number back:

$$\mathbf{p}^c = z \mathbf{K}^{-1} \tilde{\mathbf{x}}_s. \quad (3.2.1)$$

Here $\tilde{\mathbf{x}}_s$ is the homogeneous pixel (third component 1), $\mathbf{K}$ the intrinsics from lecture 01, and $\mathbf{p}^c$ the point in the camera frame.

Proposition 3.2.2 (Projection is not invertible). *The third component of $\mathbf{K}^{-1}\tilde{\mathbf{x}}_s$ is 1, so the free scalar z in (3.2.1) is the depth in meters along the optical axis. Given only $\mathbf{x}_s$, the set of 3D points that project to it is the whole ray*

$$\{z \mathbf{K}^{-1}\tilde{\mathbf{x}}_s : z > 0\}.$$

Proof. Third row of $\mathbf{K}$ is $(0, 0, 1)$, hence $\mathbf{K}\mathbf{p}^c = (*, *, p_z^c)^\top$ and $\mathbf{p}^c$ projects to $\tilde{\mathbf{x}}_s$ iff $\mathbf{K}\mathbf{p}^c = \lambda\tilde{\mathbf{x}}_s$ with $\lambda = p_z^c$ (homogeneous division). Solving, $\mathbf{p}^c = \lambda\mathbf{K}^{-1}\tilde{\mathbf{x}}_s$ and $\lambda = z$. Conversely every $z\mathbf{K}^{-1}\tilde{\mathbf{x}}_s$ satisfies $\mathbf{K}(z\mathbf{K}^{-1}\tilde{\mathbf{x}}_s) = z\tilde{\mathbf{x}}_s \sim \tilde{\mathbf{x}}_s$. $\square$

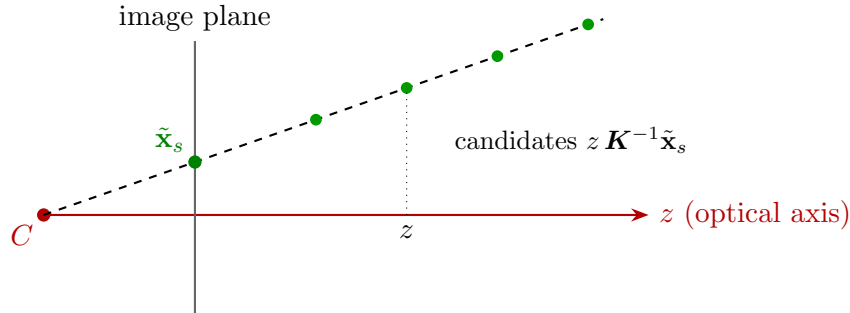


Figure 1: Back-projected ray (redrawn from slide 6; side view, y up). Every green point projects to the same pixel.

Intuition

It is like knowing the *direction* in which a laser pointer is aimed but not how far away the dot lands. The information was *destroyed at capture*, not merely hidden: no clever algorithm can un-destroy it, so every method that produces z is either measuring something extra or importing an assumption. “Producing z at every pixel is the whole of this lecture.”

3.2.3 Scale-Depth Ambiguity (slides 7)

Proposition 3.2.3 (Global scale is unobservable from one image). *Let $\{\mathbf{p}_j\}$ be a scene in the camera frame and $s > 0$. The scaled scene $\{s\mathbf{p}_j\}$ produces exactly the same image.*

Proof. $\pi(\mathbf{K} s\mathbf{p}) = \pi(s \mathbf{K}\mathbf{p}) = \pi(\mathbf{K}\mathbf{p})$, because the homogeneous division cancels s . $\square$

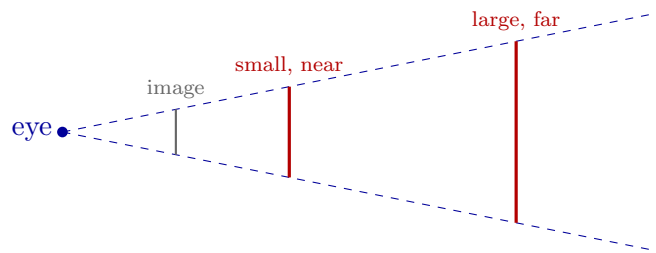


Figure 2: Scale-depth ambiguity (schematic after slide 7): a small near object and an object twice as large at twice the distance subtend the same angle.



Figure 3: two photos of a giant red bench, the second with a bicycle placed on it

Intuition

A small close object looks the same as a much larger one further away. The giant bench in the slide's photographs is a real-life example: on its own it reads as a normal bench, and only the bicycle on top reveals its true size. That bicycle is exactly a *prior about sizes* – the theme of the rest of the lecture.

3.3 Three Ways to Pay for Depth (*slides 8–12*)

Time, a baseline, a prior. No method returns depth for free. Each buys it with a different resource, and the resource decides what the result is good for.

3.3.1 Two Measurement Principles (*slides 9*)

Definition 3.3.1 (Time of flight and triangulation). Two independent choices: the *principle* decides what is measured; the *illumination* decides only whether the sensor brings its own light (*active*) or uses ambient light (*passive*).

- **Time of flight (ToF)**: measure the delay of emitted light.
- **Triangulation**: measure the angular/pixel difference between two viewpoints separated by a known geometry.

	time of flight	triangulation
active (own light)	scanning laser, flash range camera	projected-pattern camera, active stereo
passive (ambient light)	<i>empty</i> : timing needs a known emission	stereo with a known baseline, structure from motion

Table 3: The two-by-two of depth sensing (slide 9).



Figure 4: left: speckle pattern of a projected-pattern camera with a stereo camera overlaid; right: a Hesai spinning LiDAR

Intuition

The empty cell is instructive: to time light you must *know when it left*, and ambient light has no known departure time. Also, “active” only helps with texture or timing – it does not change the physics: a projected pattern camera is still triangulation, it just paints texture onto blank walls so matching works.

3.3.2 Triangulation From Two Views (*slides 10*)

Two rectified cameras sit a baseline b apart. A scene point $\mathbf{p} = (x, y, z) \in \mathbb{R}^3$ appears at horizontal pixel offsets u_L and u_R (relative to each principal point), and similar triangles give the depth.

Theorem 3.3.2 (Depth from disparity). *For two rectified cameras with focal length f (in pixels) and baseline b (in meters), the depth of a point with disparity $d = u_L - u_R$ is*

$$z = \frac{fb}{d}.$$

Proof. By the pinhole model $u_L = fx/z$ for the left camera, and since the right camera is displaced by b along x , $u_R = f(x - b)/z$. Subtracting, $d = u_L - u_R = fb/z$. (The slide draws the image plane behind the pinhole, which produces the minus signs in its two similar-triangle equations; the resulting formula is the same.) $\square$

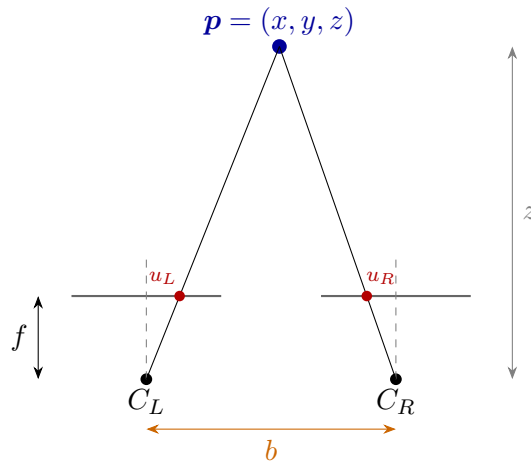


Figure 5: Rectified stereo (redrawn from slide 10, top view). With $f = 1$, $b = 3$, $z = 4$ as drawn: $u_L = 0.4$, $u_R = -0.35$, $d = 0.75 = fb/z$.

Corollary 3.3.3 (Unknown baseline). *Disparity is inverse depth scaled by fb : $1/z = d/(fb)$. If b is unknown (a moving camera), depth is known only up to one global scale.*

Example 3.3.4. For $f = 700$ px and $b = 0.5$ m, $fb = 350$ px m and $z = 350/d$: $d = 70$ px means 5 m, $d = 17.5$ px means 20 m, $d = 7$ px means 50 m.

Intuition

Where meters come from: b is in meters, so z is in meters – the baseline is the source of scale. Hold a finger at arm's length and blink one eye at a time: the finger jumps a lot against the background; a distant tree barely moves. That jump *is* the disparity, and it is large for near things (large $1/z$). **Matching** is the price: d requires finding the *same* point in both images, which fails on textureless or repetitive surfaces and wherever a surface is visible to one view only. Because disparity is fb times inverse depth, an unknown rig only leaves an unknown *scale* in disparity, which is why affine invariance in *disparity* suits data whose rig is unknown (used in §3.7).

3.3.3 The Three Currencies (slides 11)

3.3.4 The Single-Image Claim (slides 12)

The usual motivation for single-image depth is that it bypasses *all* these limitations. The slide's verdict: a single image does bypass the **cost**, the **sparsity** and the **hardware**, but it cannot

pay with	method	gives	costs
time	emit a pulse and time the return (LiDAR)	metric depth, directly	sparse, expensive, returns missing
a baseline	stereo, baseline known	metric depth, dense where matching works	a calibrated second camera; error growing as z^2
computation	structure from motion, baseline unknown	shape up to one global scale	a moving camera and a rigid scene
a prior	assumptions learned from data	dense, from one image, cheap	not metric, and only as good as the prior

Table 4: The three currencies (slide 11).

bypass the **physics**, because it *never measured anything*. The result is therefore an *ambiguous* quantity (same ray picture as Figure 1).

Intuition

Single-image depth is not a cheaper LiDAR; it is a different kind of object. It converts the question “how far?” into “how far, *given how scenes usually look?*” – a guess with an informed prior. Keeping this distinction in mind explains every failure mode in §3.8.

3.4 Inside a LiDAR (*slides 13–17*)

Paying with time. Outdoor depth labels are produced by a scanning laser, and their error sources are inherited by any network trained on them – so we need to know how a LiDAR fails.

3.4.1 Pulsed Time of Flight (*slides 14*)

A short pulse is emitted and a clock runs until the echo returns. The light covers the distance twice, so the interval is halved:

$$r = \frac{ct}{2}. \quad (3.4.1)$$

Each beam leaves in a known direction, so a return is a *direction plus a range*; the energy that comes back is reported as *intensity*. Pulse widths are a few nanoseconds (typically 5–10 ns, about 4 ns in modern full-waveform scanners). Every other part of the sensor exists to make the interval t reliable.

Example 3.4.1. An echo from 50 m returns after $t = 2 \cdot 50 / (3 \cdot 10^8) \approx 333$ ns. A timing error of 1 ns corresponds to $c \delta t / 2 \approx 15$ cm of range. A 5 ns pulse is $c \cdot 5 \text{ ns} \approx 1.5$ m long in space.

Intuition

It is echolocation with light: shout and time the echo. The factor $\frac{1}{2}$ is just the round trip. Because c is so large, centimeter accuracy demands sub-nanosecond timing – which is why timing electronics dominate the design.

3.4.2 Structured Sparsity (*slides 15*)

Azimuth resolution is set by the spin rate; elevation resolution is fixed in hardware. The two differ, so the sampling is *anisotropic*.

Definition 3.4.2 (Scan-line spacing). For a vertical angular pitch $\Delta\theta_{\text{el}}$ (radians), the spacing

between adjacent scan lines at range r is

$$s(r) \approx r \Delta\theta_{\text{el}}.$$

Example 3.4.3 (Velodyne VLP-16). At 5 Hz the sensor samples every 0.1° in azimuth; at 20 Hz every 0.4° – spinning faster costs horizontal resolution. Vertically it is stuck at $\Delta\theta_{\text{el}} = 2.0^\circ = 0.0349 \text{ rad}$, so the scan lines are 0.70 m apart at 20 m and 1.75 m apart at 50 m. A pedestrian at 50 m is crossed by one scan line or by none. (For comparison, the azimuth gap at 50 m is only 0.09 m at 5 Hz and 0.35 m at 20 Hz.)

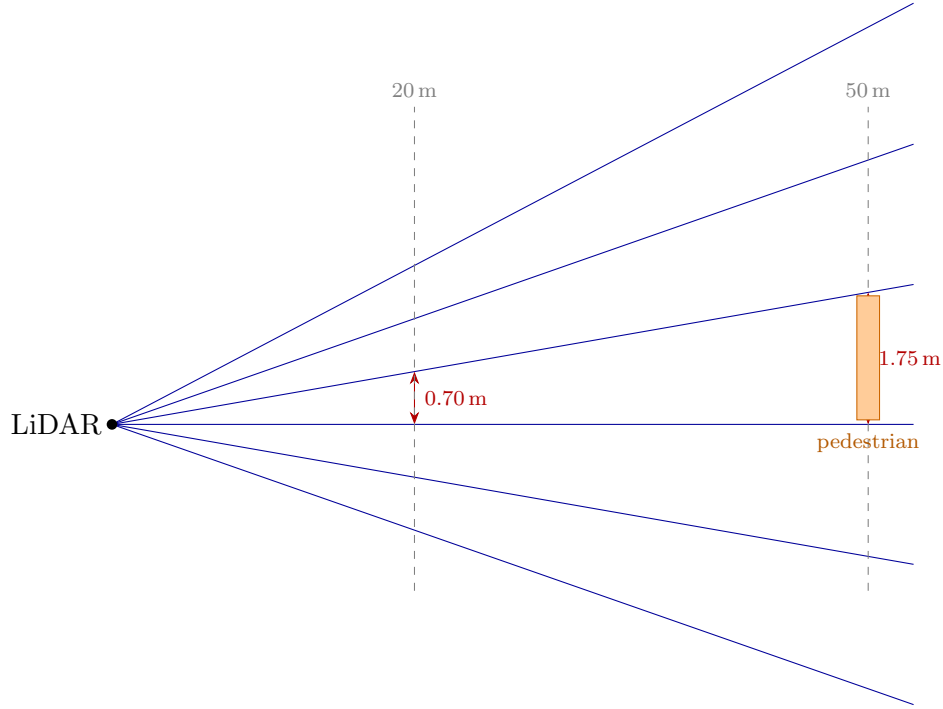


Figure 6: Original figure (not on the slides): VLP-16 elevation fan with a 2° pitch, side view. Vertical axis exaggerated: 1 cm = 1 m vertically but 1 cm = 5 m along range. At 50 m a 1.7 m pedestrian fits between two consecutive scan lines and is missed.

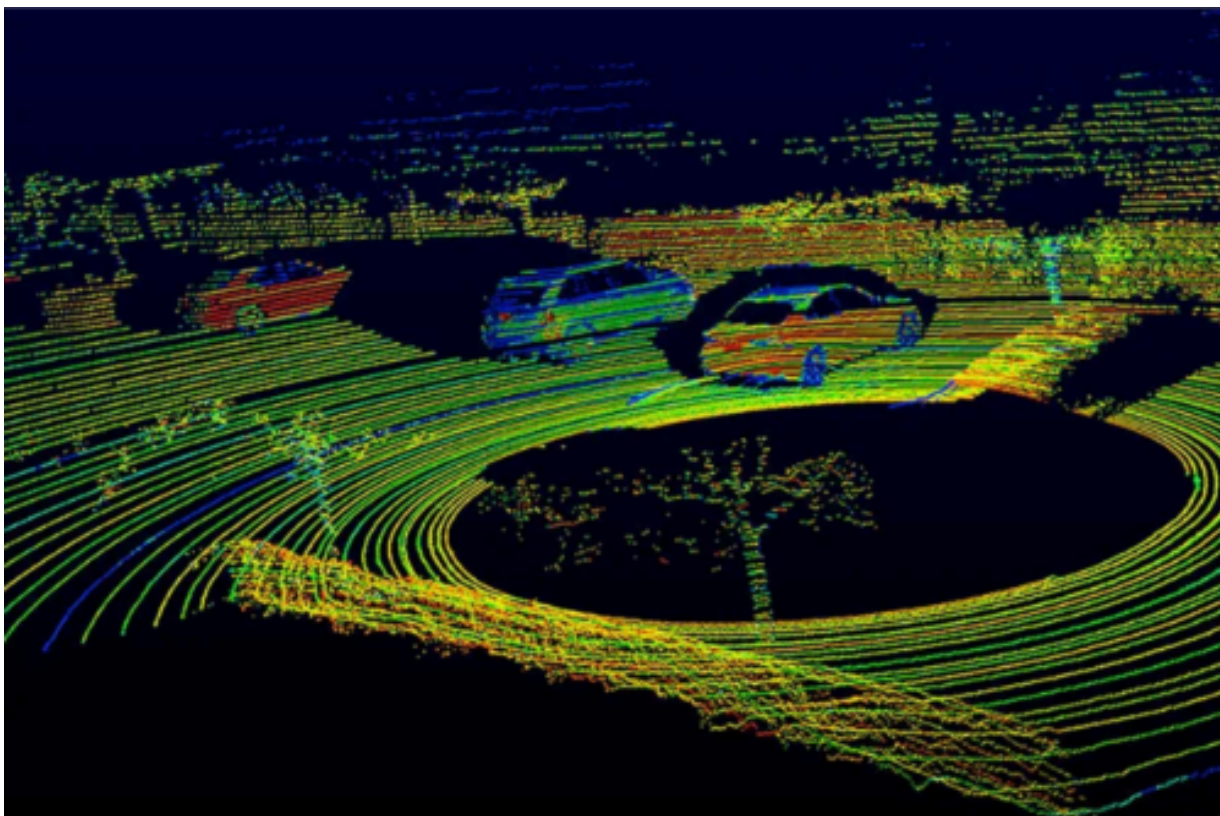


Figure 7: colored LiDAR point cloud of a parking lot: concentric ring-like scan lines with gaps between them

Intuition

Picture the grooves of a vinyl record. Samples are dense *along* a groove (azimuth) but grooves are far apart (elevation). Moving away from the sensor, the grooves spread linearly with range, while points along a groove stay comparatively tight. So a point cloud thins with distance along *one axis* much faster than the other, and the gaps form horizontal stripes.

3.4.3 Limitations of a LiDAR Point Cloud (*slides 16*)

- **No color and no semantics.**
- **No measurement where there was no return**, which is *distinct from a zero depth*: “I saw nothing” is not “the surface is at distance 0”.
- **No sample between scan lines**, and that gap grows linearly with range.
- **Points that lie on no surface**: where one pulse straddles a silhouette, the echo mixes foreground and background into a single (wrong) range.

Intuition

The last item is the LiDAR analogue of an out-of-focus edge: half the beam hits the near object, half the far wall, and the reported range is somewhere in the empty air between them. Such “flying” points are a systematic error that a network trained on LiDAR labels will inherit at every object boundary.

3.4.4 Two Error Laws (slides 17)

The two principles fail in different ways, and the difference decides where each is worth using.

$$\text{time of flight: } \delta r = \frac{c \delta t}{2}, \quad \text{triangulation: } \delta z \approx \frac{z^2 \delta d}{f b}.$$

Proposition 3.4.4 (Triangulation error law). *A disparity error δd produces a depth error $\delta z \approx z^2 \delta d / (fb)$: quadratic in range, even for a perfect sensor.*

Proof. From Theorem 3.3.2, $z = fb/d$, so $|dz/dd| = fb/d^2 = z^2/fb$. Multiply by δd . □

The timing floor $c \delta t / 2$ is *independent of range*. What degrades with range is the *precision*: the returned power falls as $1/r^2$ or $1/r^4$ (depending on whether the target is extended or smaller than the beam – standard radar/LiDAR-equation background), and a weaker return is timed less precisely.

Example 3.4.5 (Numbers from the slide). With $f = 700$ px, $b = 0.5$ m and quarter-pixel matching ($\delta d = 0.25$ px), $fb = 350$ and

$$\delta z(5 \text{ m}) = \frac{25 \cdot 0.25}{350} \approx 2 \text{ cm}, \quad \delta z(20 \text{ m}) \approx 29 \text{ cm}, \quad \delta z(50 \text{ m}) \approx 1.8 \text{ m}.$$

The laser is exact where it lands, but at 50 m its scan lines are 1.75 m apart, so a pedestrian is crossed at most once.

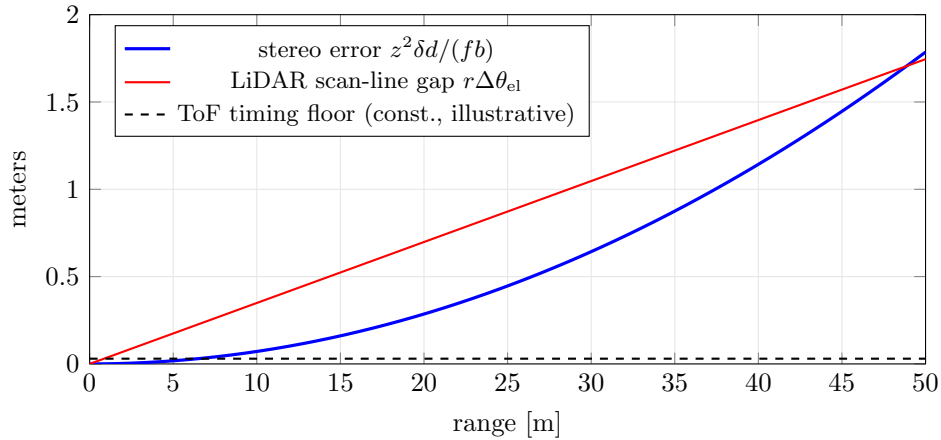


Figure 8: Original figure (not on the slides), from the slide's numbers: $f = 700$ px, $b = 0.5$ m, $\delta d = 0.25$ px, $\Delta \theta_{el} = 2^\circ$. The ToF floor level (3 cm) is only illustrative.

Intuition

The two sensors are *complementary*: LiDAR is precise where it samples but leaves holes; stereo samples everywhere but each sample degrades quadratically. This is why the remaining sections either *fuse* the two or *replace the laser with a prior*.

3.5 From Point Cloud to Depth Image (slides 18–19)

Paying with time: from cloud to depth map. A LiDAR produces a point cloud. Converting it to a depth map needs the extrinsic calibration, the intrinsics (lecture 01), and a rule for resolving points that fall in the same pixel.

3.5.1 Projecting a Cloud Into the Image (*slides 19*)

The extrinsics are applied, followed by the projection from lecture 01:

$$\tilde{\mathbf{x}}_s = \pi\left(\mathbf{K} \begin{bmatrix} \mathbf{R}_{cl} & \mathbf{t}_{cl} \end{bmatrix} \tilde{\mathbf{p}}^l\right), \quad (3.5.1)$$

where $\pi(\cdot)$ is the homogeneous division and $\mathbf{p}^l$ a point in the LiDAR frame.

Definition 3.5.1 (Depth map). The number written into the pixel is the z component in the *camera* frame. The array of them is the depth map $\mathbf{D} : \Omega \rightarrow \mathbb{R}_{>0}$,

$$\mathbf{D}[\mathbf{x}_s] = (\mathbf{R}_{cl} \mathbf{p}^l + \mathbf{t}_{cl})_z.$$

The slide stresses that this *differs from the range the LiDAR measured*.

Proposition 3.5.2 (Depth versus range). *If θ is the angle between the ray of pixel $\mathbf{x}_s$ and the optical axis, then $z = r \cos \theta$, where $r = \|\mathbf{p}^c\|$ is the range; equivalently $r = z \|\mathbf{K}^{-1} \tilde{\mathbf{x}}_s\| \geq z$.*

Proof. By (3.2.1), $\mathbf{p}^c = z \mathbf{K}^{-1} \tilde{\mathbf{x}}_s$ with $\mathbf{K}^{-1} \tilde{\mathbf{x}}_s = (a, b, 1)^\top$; so $r = z\sqrt{a^2 + b^2 + 1}$. The unit direction has z -component $1/\sqrt{a^2 + b^2 + 1} = \cos \theta$. $\square$

Example 3.5.3. A return with range $r = 20$ m at 30° off the optical axis is written as $z = 20 \cos 30^\circ \approx 17.3$ m in the depth map.

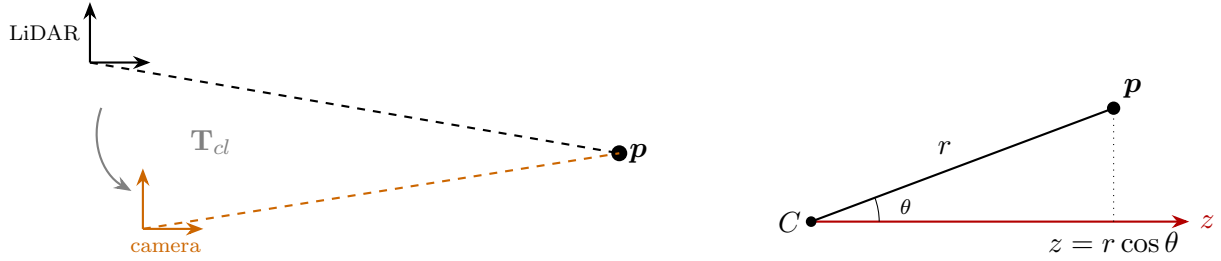


Figure 9: Left: LiDAR and camera frames related by $\mathbf{T}_{cl}$ (redrawn from slide 19). Right: range r versus depth z (original figure).

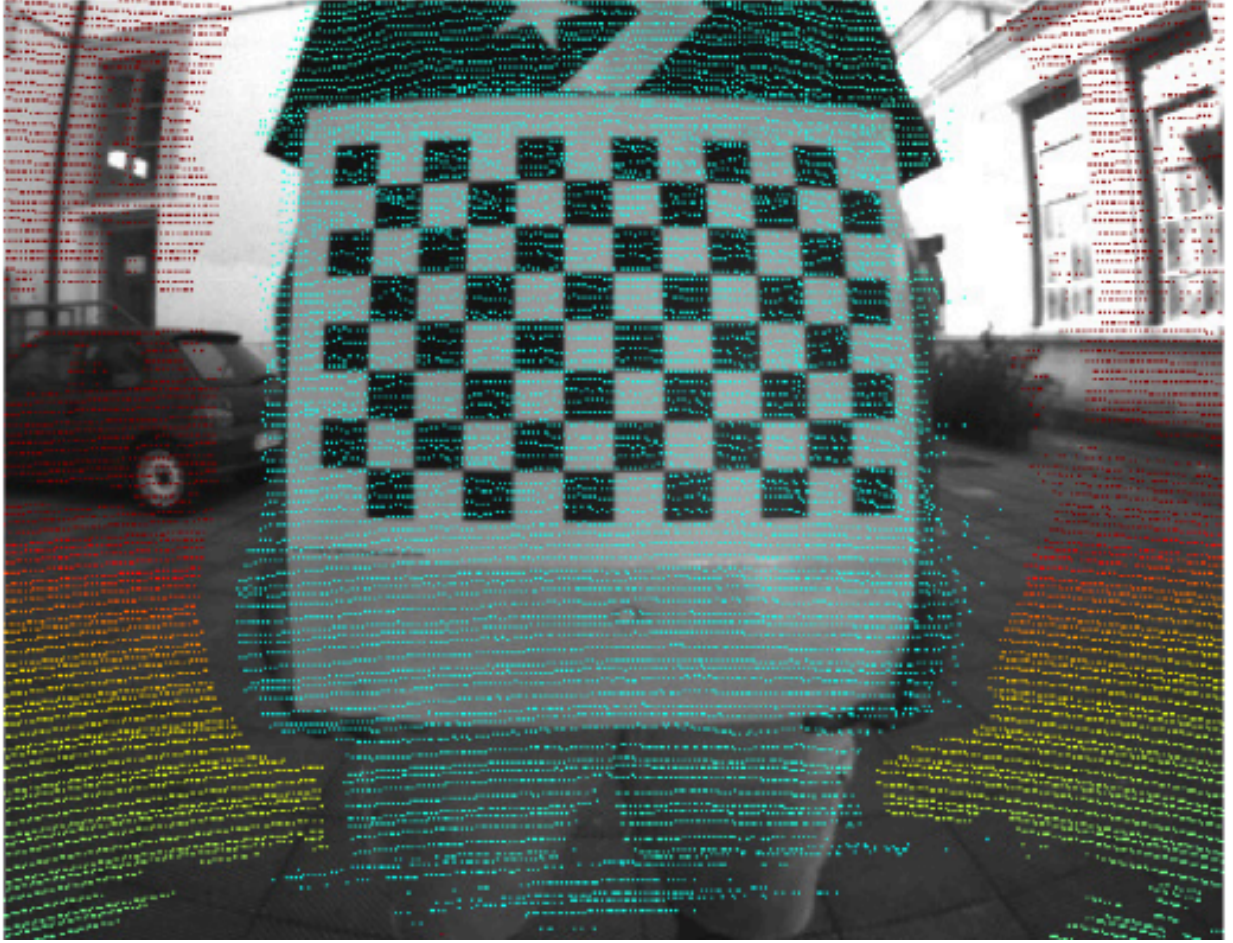


Figure 10: grayscale image of a checkerboard target with LiDAR points projected onto it, colored by depth

Algorithm 3.5.4 (LiDAR cloud to sparse depth map). Input: points $\{p_j^l\}$, intrinsics $\mathbf{K}$, extrinsics $(\mathbf{R}_{cl}, \mathbf{t}_{cl})$, image size.

1. Initialize $\mathbf{D} \leftarrow 0$ everywhere (“no measurement”).
2. For each point: $\mathbf{p}^c = \mathbf{R}_{cl}\mathbf{p}^l + \mathbf{t}_{cl}$; skip if $p_z^c \leq 0$ (behind the camera).
3. $\tilde{\mathbf{x}}_s = \pi(\mathbf{K}\mathbf{p}^c)$, round to a pixel; skip if outside the image.
4. If $\mathbf{D}[\mathbf{x}_s] = 0$ or $p_z^c < \mathbf{D}[\mathbf{x}_s]$, set $\mathbf{D}[\mathbf{x}_s] \leftarrow p_z^c$.

Intuition

Step 4 is one common rule for “points that fall in the same pixel” (keep the nearest, as a z-buffer does); the slide only says such a rule is needed. Valid depths are strictly positive ($\mathbf{D} : \Omega \rightarrow \mathbb{R}_{>0}$), so 0 can safely mark “no return” without colliding with a genuine measurement – matching the earlier warning that a missing return is not a zero depth. Also remember that the result is *sparse*: most pixels stay 0.

3.6 From Recovery to Prediction (*slides 20–22*)

Paying with a prior. One image does not determine depth. People judge it anyway, so they are supplying an assumption. This section turns that observation into a prior.

3.6.1 Monocular Cues in Human Vision (*slides 21*)

Not all 3D structures are equally likely, and human vision exploits that regularity through seven classic cues:

1. linear perspective,
2. relative size / position,
3. texture gradient,
4. occlusion (the occluded object is farther),
5. aerial perspective,
6. light and shadows,
7. blur and defocus.

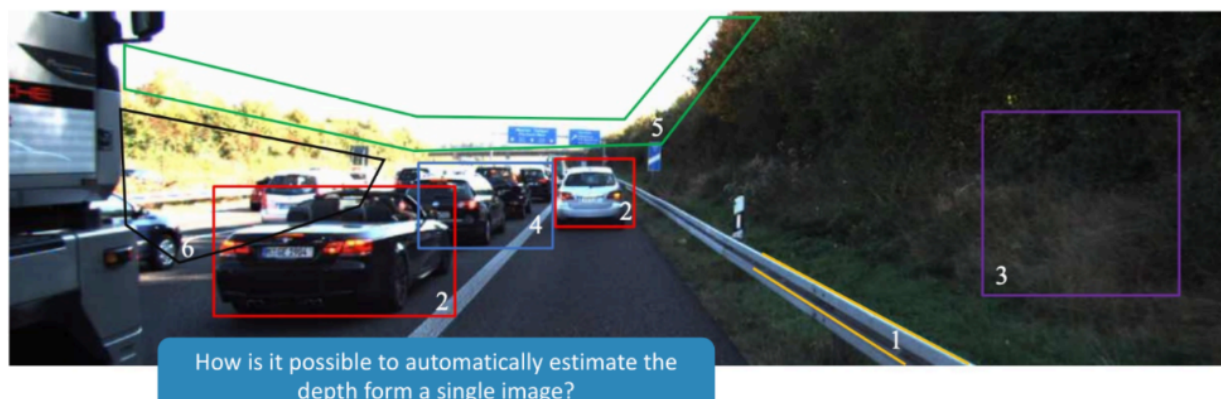


Figure 11: driving image (highway scene) with the seven cues outlined and numbered

Intuition

None of these is a measurement. Each is an *assumption about how scenes are usually arranged*: parallel lines meet, near things are big, far things are hazy. They work because the world is not a uniformly random set of 3D structures.

3.6.2 Depth Prediction as a Prior (*slides 22*)

Proposition 3.6.1 (Bayesian view of single-image depth).

$$p(\mathbf{D} \mid \mathbf{I}) \propto p(\mathbf{I} \mid \mathbf{D})p(\mathbf{D}).$$

The likelihood term is nearly flat along each ray (this is the ill-posedness of §3.1), so the posterior is dominated by the prior $p(\mathbf{D})$. Hence single-image depth is predicted rather than recovered, and a method is a claim about which scenes are likely.

Intuition

The likelihood asks: “does this depth map explain the picture?” Along a ray, every depth explains the pixel equally well, so the picture alone cannot vote. The only thing left to break ties is what depth maps *usually look like* – and people reading depth from a single photograph is everyday evidence that such a prior exists and is learnable. Where the learned one fails is the topic of §3.8.

3.7 Cues in Closed Form (*slides 23–26*)

The prior, written in closed form. Vanishing points, apparent size and vertical position each follow in one line from the projection equation.

3.7.1 Vanishing Points (*slides 24*)

Parallel 3D lines converge in the image; the horizon is the image of the line at infinity of the ground plane. A vanishing point is the image of a *direction*, not of a location; every set of parallel lines has its own, and for a given plane they all lie on one line.

Proposition 3.7.1 (Vanishing point of a direction). *Let $\mathbf{d} \in \mathbb{R}^3$ be a 3D direction. Its image $\mathbf{v}$ satisfies*

$$\lambda \mathbf{v} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}] \begin{pmatrix} \mathbf{d} \\ 0 \end{pmatrix} = \mathbf{K} \mathbf{R} \mathbf{d}.$$

Proof. A direction is a point at infinity, homogeneous coordinates $(\mathbf{d}, 0)$. In the product $[\mathbf{R} \mid \mathbf{t}](\mathbf{d}, 0)^\top = \mathbf{R}\mathbf{d} + \mathbf{t} \cdot 0$ the translation is multiplied by zero and drops out. $\square$

Corollary 3.7.2. *The horizon moves with camera **pitch and roll** only; translation does not move it.*

Example 3.7.3 (Horizon row for a pitched camera – extra worked case). Take zero roll, image y pointing down, and pitch the camera up by θ . The level forward direction has camera-frame coordinates $\mathbf{R}\mathbf{d} = (0, \sin \theta, \cos \theta)^\top$, so $\mathbf{K}\mathbf{R}\mathbf{d}$ gives the horizon row $y_h = c_y + f \tan \theta$: pitching up moves the horizon *down* in the image.

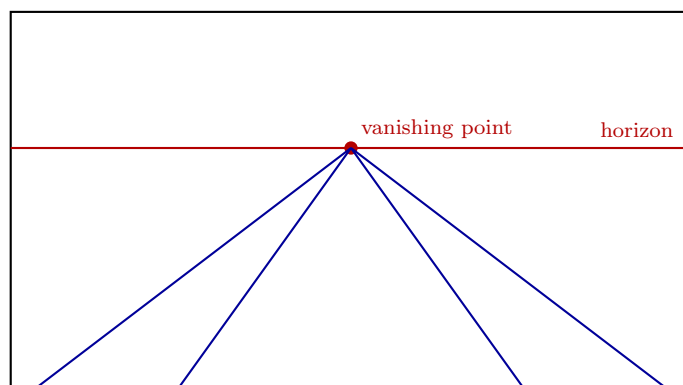


Figure 12: Original schematic (the slide shows a photograph, see below): parallel lines on the ground plane meet on the horizon.

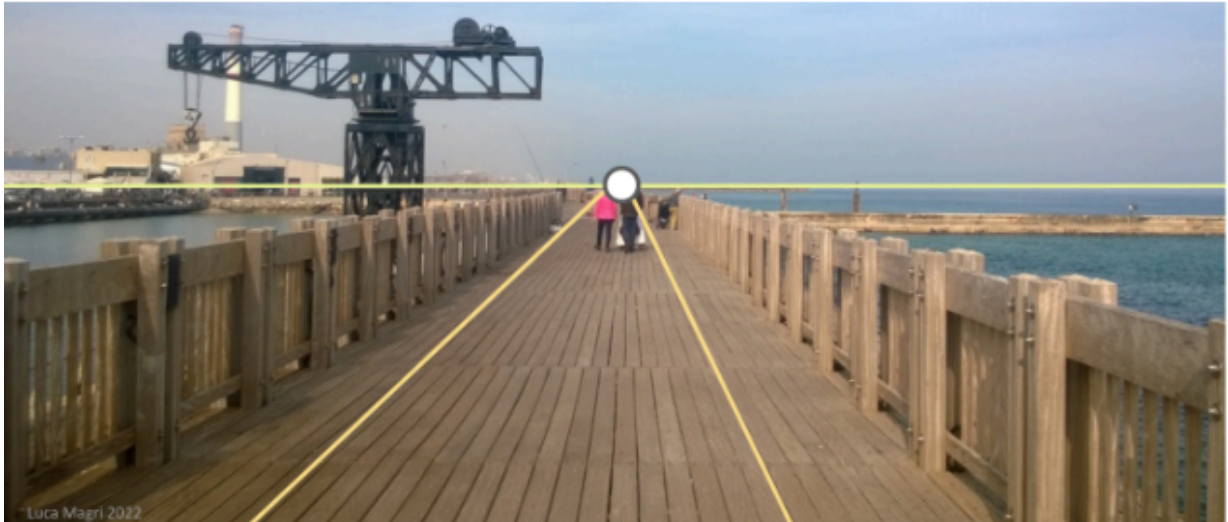


Figure 13: photo of a wooden pier with yellow lines converging to a vanishing point on the horizon

Intuition

The translation drops out because a direction has no position: walking sideways does not change where the rails converge, only tilting the camera does. The slide adds a warning to remember: *a network later on does not respect this* (§3.8).

3.7.2 Apparent Size (*slides 25*)

Proposition 3.7.4 (Depth from apparent size). *If the true height H of an object is known and its image height is h (in pixels), then*

$$z = \frac{f H}{h}.$$

Proof. The camera center, the top and the bottom of the object form a triangle similar to the one formed by the center and the image of the object: $h/f = H/z$. $\square$

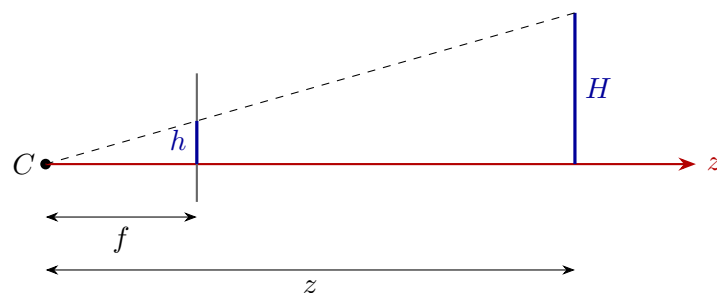


Figure 14: Apparent size (redrawn from slide 25, side view).

Example 3.7.5. A car $H = 1.5$ m tall that appears $h = 50$ px tall with $f = 700$ px is at $z = 700 \cdot 1.5/50 = 21$ m.

Intuition

This works on driving data because almost everything in it is a car, a truck or a pedestrian, and those have known sizes: the *prior about sizes* is doing the work. It fails for objects of unknown size – the giant bench of §3.1 again.

3.7.3 Vertical Position (*slides 26*)

Proposition 3.7.6 (Depth from the ground contact point). *On a flat ground plane with known camera height Y , an object touching the ground at image row y_s (horizon row y_h) has depth*

$$z = \frac{fY}{y_s - y_h}.$$

Proof. For a level camera, the ground point has camera-frame vertical coordinate Y (below the camera) and depth z ; projection gives $y_s - y_h = fY/z$. Solve for z . $\square$

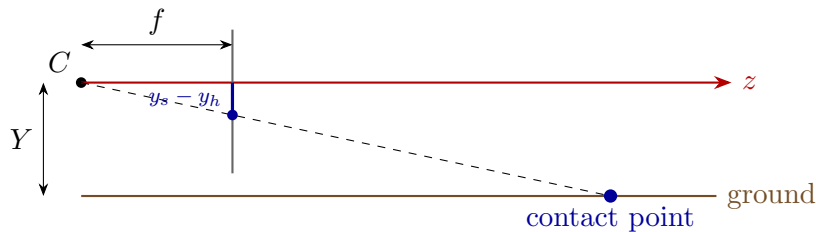


Figure 15: Vertical position (redrawn from slide 26, side view; horizon = optical axis for a level camera).

Corollary 3.7.7 (Sensitivity). *Differentiating, $\delta z = z^2 \delta y / (fY)$.*

Example 3.7.8. $f = 700$ px, $Y = 1.65$ m, contact point 40 px below the horizon: $z = 700 \cdot 1.65 / 40 \approx 28.9$ m. A one-pixel error moves this by $z^2 / (fY) \approx 0.73$ m.

Intuition

No knowledge of the object is required, only its contact point with the ground; but *the assumptions carry the estimate*: flat ground, known camera height, known horizon. Note the resemblance to Proposition 3.4.4: the error also grows as z^2 , with the camera height Y playing the role of a baseline (a remark of these notes, not on the slide). Cues near the horizon are extremely sensitive.

3.8 Supervised Single-View Depth (*slides 27–35*)

Paying with a prior, part two. Fitting the prior requires an architecture, a loss that respects the scale ambiguity, and measured depth data.

3.8.1 The Supervised Setup (*slides 28*)

Definition 3.8.1 (Supervised depth learning). A training set of image / depth-map pairs, and a network from $3 \times H \times W$ to $1 \times H \times W$ trained with a per-pixel loss:

$$\text{TR} = \{(\mathbf{I}_i, \mathbf{D}_i)\}_{i=1}^N, \quad \hat{\mathbf{D}} = f_\theta(\mathbf{I}).$$

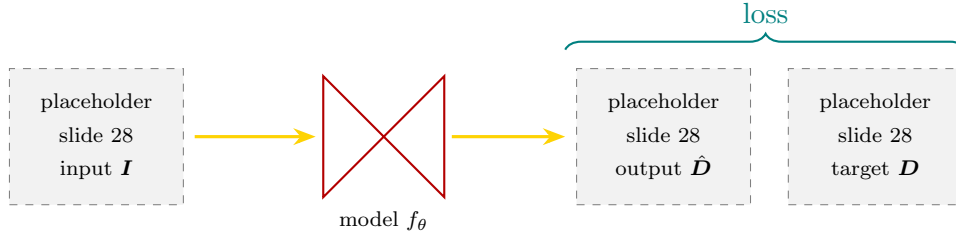


Figure 16: The supervised setup (redrawn from slide 28; the two depth maps are photographs to copy from the slide).

Intuition

Data, network and per-pixel loss are the whole recipe; the slide’s summary is that *every remaining design decision is inside the loss* (what counts as a correct prediction when scale is unobservable).

3.8.2 Three Datasets (*slides 29*)

Each of the three sits in a different cell of the principles table: a projected-pattern camera, a scanning laser, and triangulation with no known baseline.

- **NYU Depth v2**: indoor, from a structured-light depth camera, with its raw depth *in-painted* to give a dense label.
- **KITTI**: driving, LiDAR projected into the camera as in §3.4.
- **MegaDepth**: internet photo collections through structure from motion and multi-view stereo. Dense, varied, and *scale-ambiguous by construction*.

These are the $\mathbf{D}_i$ of the previous slide. Each inherits the error sources of the sensor that produced it, and the last carries no metric scale at all – which is what the next two methods are designed around.

Intuition

Every dataset is a *sensor’s opinion* of depth. NYU inherits the holes and inpainting of a structured-light camera, KITTI inherits LiDAR sparsity and flying points, MegaDepth inherits SfM’s arbitrary scale. Datasets with “no scale & shift” or “ordinal” labels cannot be trained on with a plain L2 loss: the loss has to be told which ambiguity to ignore.

3.8.3 The Scale-Invariant Loss (*slides 30*)

The direct choice, $\mathcal{D}(\mathbf{D}, \hat{\mathbf{D}}) = \|\mathbf{D} - \hat{\mathbf{D}}\|_2^2$, penalizes the network for a global scale the image does not determine. What is required is a loss invariant to that scale:

$$\mathcal{D}(\mathbf{y}, \mathbf{y}^*) = \mathcal{D}(s\mathbf{y}, \mathbf{y}^*) \quad \forall s > 0.$$

Dataset	Annotation	Depth type	Accuracy	Diversity	# images
DIML Indoor	RGB-D	metric	medium	medium	220K
MegaDepth	SfM	no scale	medium	medium	130K
ReDWeb	stereo	no scale & shift	medium	high	3600
WSVD	stereo	no scale & shift	medium	high	1.5M
3D Movies	stereo	no scale & shift	medium	high	75K
DIW	user clicks	ordinal pair	low	high	496K
ETH3D	laser	metric	high	low	454
Sintel	synthetic	(metric)	high	medium	1064
KITTI	laser/stereo	metric	medium	low	93K
NYUDv2	RGB-D	metric	medium	low	407K
TUM-RGBD	RGB-D	metric	medium	low	80K

Table 5: Trimmed transcription of the dataset table on slide 29 (the indoor/outdoor/dynamic/video/dense check-mark columns are omitted; copy the full table from the slide if needed).

Definition 3.8.2 (Eigen–Puhresch–Fergus scale-invariant loss (NIPS 2014)). Mean squared error in log space, plus a per-image offset:

$$\mathcal{D}(\mathbf{y}, \mathbf{y}^*) = \frac{1}{n} \sum_{i=1}^n (\log y_i - \log y_i^* + \alpha)^2, \quad \alpha = \frac{1}{n} \sum_{i=1}^n (\log y_i^* - \log y_i).$$

Lemma 3.8.3 (Closed form and invariance). Write $d_i = \log y_i - \log y_i^*$. Then $\alpha = -\bar{d}$ minimizes the expression over all offsets, and

$$\mathcal{D}(\mathbf{y}, \mathbf{y}^*) = \frac{1}{n} \sum_i d_i^2 - \frac{1}{n^2} \left(\sum_i d_i \right)^2 = \text{Var}(d_i).$$

Moreover $\mathcal{D}(s\mathbf{y}, \mathbf{y}^*) = \mathcal{D}(\mathbf{y}, \mathbf{y}^*)$ for every $s > 0$.

Proof. Differentiating $\sum_i (d_i + \alpha)^2$ in α and setting to zero gives $\alpha = -\bar{d}$; substituting yields $\frac{1}{n} \sum (d_i - \bar{d})^2$, which is the stated variance. Replacing $\mathbf{y}$ by $s\mathbf{y}$ adds the constant $\log s$ to every d_i ; a variance does not change under a constant shift. $\square$

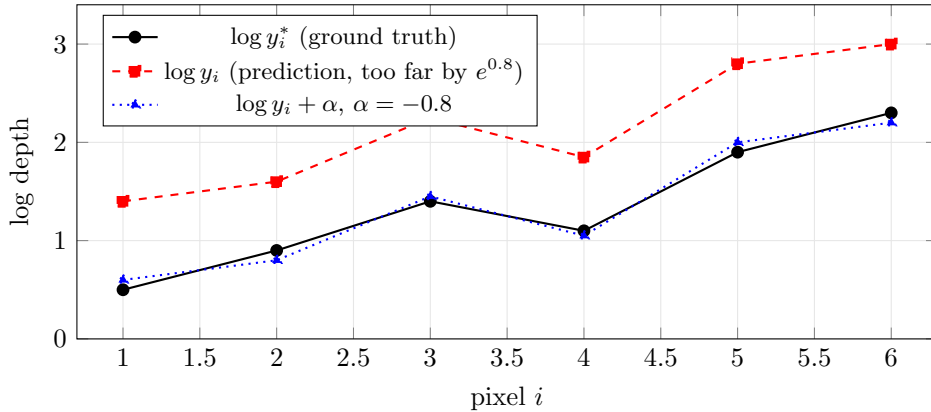


Figure 17: Original toy example (not on the slides). The prediction has the right shape but a wrong global scale. Naive log-MSE would be ≈ 0.646 ; the scale-invariant loss, after aligning by e^α , is only ≈ 0.0058 .

Intuition

In log space a wrong *scale* becomes a constant *offset*, and the loss subtracts the best constant offset before looking at the errors. So only the *shape* of the depth map is judged. Two side remarks from the slide: log space stops far pixels (numerically large) from dominating, and $e^\alpha \mathbf{y}$ is the scalar multiple that best aligns the prediction to the ground truth – every scalar multiple of $\mathbf{y}$ scores identically.

3.8.4 Output Spaces (*slides 31*)

Five quantities a depth network can predict differ in how many unknowns are left free, and in *what the free parameters act on*.

	output	free param.	aligned by	trained this way to
1	metric depth	0	nothing	plan and measure
2	scale-invariant depth	1 (s)	one scale	share one scale per scene
3	affine-invariant disparity	2 (s, t)	scale and shift	mix datasets with no common zero
4	affine-invariant depth	2 (s, t)	scale and shift	mix datasets in depth (3.11)
5	ordinal	any monotone map	nothing	rank only

Table 6: Output spaces (slide 31). Scale-invariant: $\hat{y} \approx s y^*$; affine-invariant: $\hat{q} \approx s q^* + t$ with $q = 1/y$ for disparity.

These are routinely conflated: Eigen’s log-space is *one* parameter, affine invariance *two*. The affine row appears twice because the same pair (s, t) can act on disparity or on depth, and the two differ in 3D (next subsection). Benchmarks align the free parameters against the ground truth first, then score with δ and AbsRel, so a model is scored on *shape*, *not on meters*.

Example 3.8.4 (Standard metrics, background). AbsRel = $\frac{1}{n} \sum_i |\hat{y}_i - y_i^*|/y_i^*$, and δ_1 is the fraction of pixels with $\max(\hat{y}_i/y_i^*, y_i^*/\hat{y}_i) < 1.25$; both are computed after the alignment above.

3.8.5 Affine Invariance in Disparity and in Depth (*slides 32*)

Proposition 3.8.5 (Affine map in disparity = projective map in space). *An affine map in inverse depth,*

$$\frac{1}{z} \mapsto \frac{s}{z} + t \iff z \mapsto \frac{z}{s + tz},$$

is a nonlinear map in depth. Back-projecting the transformed depth gives $\hat{\mathbf{p}} = \mathbf{p}/(s + tp_z)$, which is a projective map of space: planes remain planes.

Proof. $s/z + t = (s + tz)/z$, so $z \mapsto z/(s + tz)$. Back-projection by (3.2.1): $\hat{\mathbf{p}} = \frac{z}{s+tz} \mathbf{K}^{-1} \tilde{\mathbf{x}}_s = \mathbf{p}/(s + tp_z)$. To see planes stay planes, let $\mathbf{a}^\top \mathbf{p} = c$ and $w = s + tp_z$. From $\hat{\mathbf{p}} = \mathbf{p}/w$ we get $\hat{p}_z = p_z/w$, hence $p_z = s\hat{p}_z/(1 - t\hat{p}_z)$ and $w = s/(1 - t\hat{p}_z)$, so $\mathbf{p} = w\hat{\mathbf{p}}$. Substituting, $s\mathbf{a}^\top \hat{\mathbf{p}} = c(1 - t\hat{p}_z)$, which is linear in $\hat{\mathbf{p}}$. $\square$

Intuition

A ground plane is therefore *tilted*, *not bent*: you can decide a scale and shift wrongly, and flat things stay flat but the world may look sheared toward or away from the camera. An affine

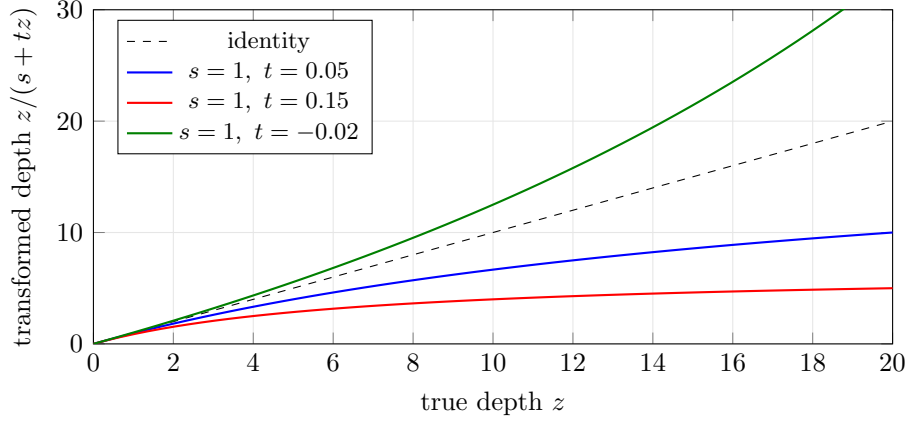


Figure 18: Original figure (not on the slides): an affine map in disparity acts on *depth* as $z \mapsto z/(s + tz)$, a compressive or expansive warp, not a rescaling.

map in depth itself would instead be $z \mapsto sz + t$, a rigid stretch and translation along the ray. The two choices give the same 2-parameter count but *different 3D geometry* – the reason the table lists the affine row twice.

3.8.6 MiDaS: Mixing the Datasets (*slides 33*)

No single dataset covers the visual world, so MiDaS trains on several. The obstacle is that they disagree about what they measure (metric, no scale, no scale and shift, ordinal, ...). Its recipe:

- predict in **disparity space** (inverse depth up to scale and shift);
- train with a **scale-and-shift-invariant loss**, where the alignment (s, t) is a least-squares fit solved *in closed form per image*;
- add a **multi-scale gradient matching** term, which is the gradient loss again, at four resolutions.

The output space is the affine-invariant *disparity* row of the table, not the affine-invariant *depth* row under it.

Lemma 3.8.6 (Closed-form alignment). *For predictions $\hat{q}_i$ and targets q_i^* , the pair*

$$(s, t) = \arg \min_{s, t} \sum_i (s \hat{q}_i + t - q_i^*)^2$$

$$s = \frac{\text{Cov}(\hat{q}, q^*)}{\text{Var}(\hat{q})}, \quad t = \bar{q}^* - s \bar{\hat{q}} \quad (\text{assuming } \text{Var}(\hat{q}) > 0).$$

Proof. Setting $\partial/\partial t = 0$ gives $t = \bar{q}^* - s \bar{\hat{q}}$. Setting $\partial/\partial s = 0$: $s \sum \hat{q}_i^2 + t \sum \hat{q}_i = \sum \hat{q}_i q_i^*$. Substituting t and dividing by n : $s(\bar{\hat{q}}^2 - \bar{\hat{q}}^2) = \bar{\hat{q}} \bar{q}^* - \bar{\hat{q}} \bar{q}^*$. $\square$

The loss is then evaluated on the residual *after* alignment. The gradient matching term, in its standard form, penalizes gradients of the aligned residual R^k at each of the $K = 4$ scales k : $\frac{1}{M} \sum_{k=1}^K \sum_i (|\nabla_x R_i^k| + |\nabla_y R_i^k|)$ (form from the original MiDaS paper; the slide only says “the gradient loss again, at four resolutions”).

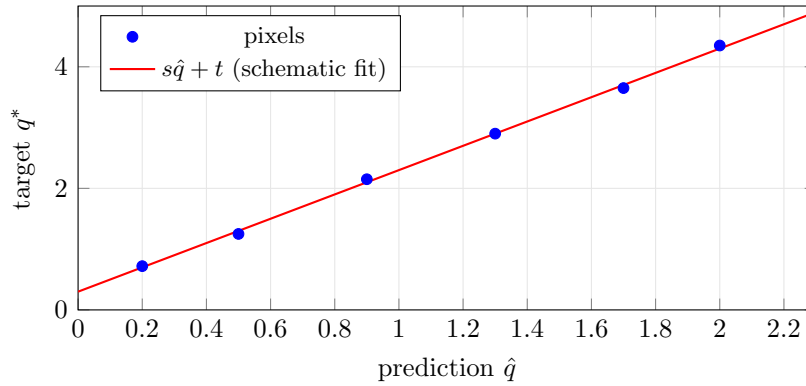


Figure 19: Original schematic (not on the slides): per-image least-squares alignment of a prediction to targets that use a different scale and zero.

Intuition

Each training image gets its *own* best-fit (s, t) before the network is judged, like a teacher who marks a student's drawing after allowing it to be enlarged and moved on the page. That is what lets one network learn from a metric laser dataset, an up-to-scale SfM dataset and a stereo-movie dataset at the same time. Working in *disparity* matters because an unknown rig only contributes an unknown scale there (Corollary 3.3.3).

3.8.7 Depth Anything V2 (slides 34)

Labeled depth data is the bottleneck, so the second version removes the requirement for it (the first version also trained on labeled real data).

- Algorithm 3.8.7** (Teacher–student training). 1. Train a **teacher** on precise *synthetic* images only, where depth is exact and complete.
 2. Use the teacher to **pseudo-label** a very large set of unlabeled real photographs.
 3. Train the **student** on those pseudo-labels.

Architecture and losses: encoder from self-supervised pretraining, decoder from dense prediction, and the *same* scale-and-shift-invariant plus gradient-matching losses as MiDaS. The output space is affine-invariant inverse depth; metric models are a separate fine-tuning step on top.

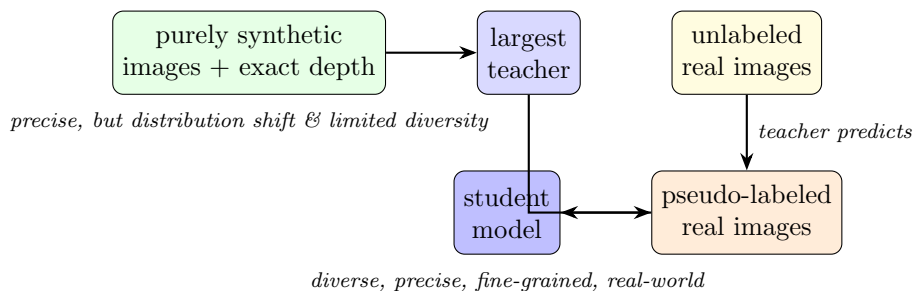


Figure 20: Depth Anything V2 pipeline (redrawn from slide 34).

Intuition

Synthetic images give *perfect* labels but look unlike the real world; real photos look right but have no labels. The teacher transfers its precision to real photos through pseudo-labels, so the student sees real-world diversity with clean, fine-grained targets. The price: the student

inherits what the teacher believes – its prior – rather than anything measured.

3.8.8 Failure on Anamorphic Art (*slides 35*)

Anamorphic street art is a flat pavement painted to look like a hole. The predicted depth map is wrong throughout, with no indication of uncertainty. Nothing has malfunctioned: the prior behaves as trained, on an input constructed to exploit it. *The prior is both the method and the failure mode.*

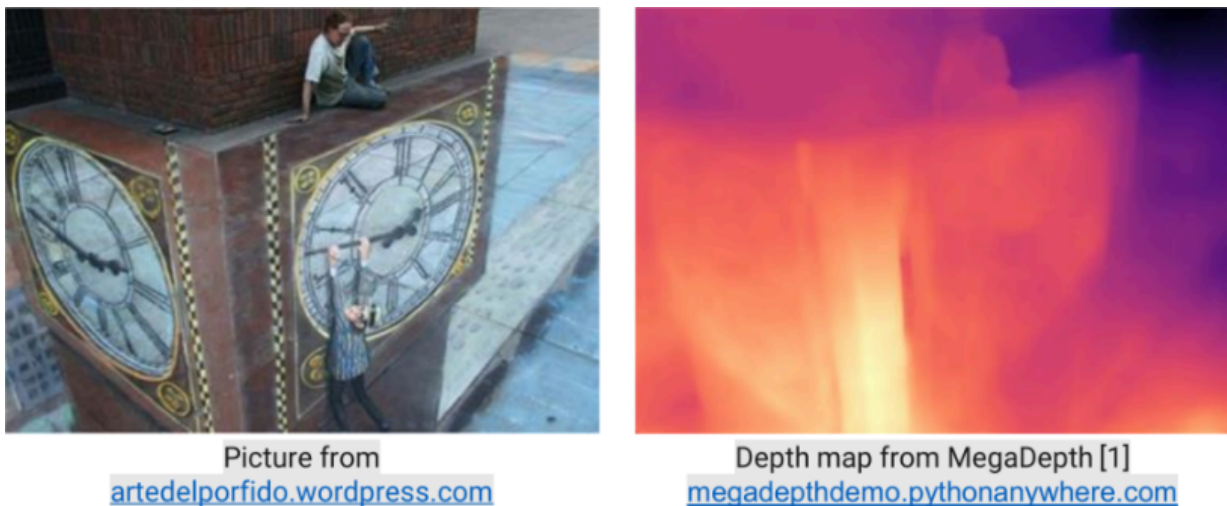


Figure 21: left: photo of a street painting of a clock tower on pavement; right: predicted depth map from MegaDepth demo

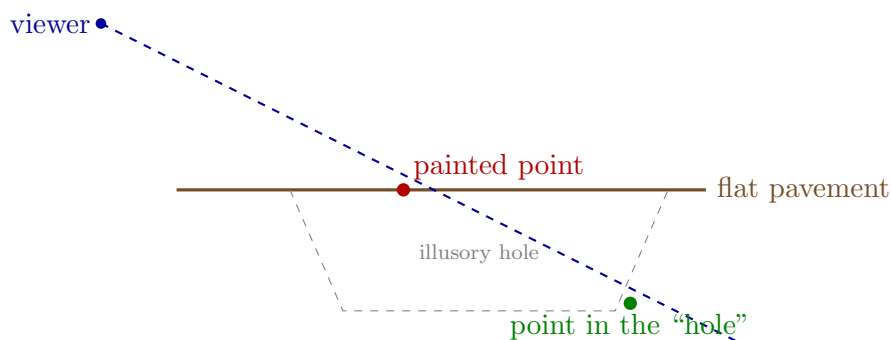


Figure 22: Original figure (not on the slides): from the painting's viewpoint the painted point on the flat ground and the point at the bottom of the imagined hole lie on the same ray (Proposition 3.2.2). One image cannot distinguish them, so the network chooses the “likely” one.

Intuition

The picture is a concrete instance of the back-projected ray: the artist chooses a flat surface whose appearance is *exactly* what a deep hole would look like from one viewpoint. Depth-from-one-image cannot know; it picks the answer its training data make most probable. Note also the “no indication of uncertainty”: the network is confidently wrong.

3.9 Cues Used by the Network (*slides 36–39*)

What the prior turns out to be. A monocular depth network is a prior fitted to a dataset. Three probes examine which image evidence that prior relies on.

3.9.1 The Pasted-Car Experiment (*slides 37*)

A car is cropped from the image and pasted back at a chosen ground contact point and a chosen scale, placing the two cues of §3.6 in conflict: both correct (position & size), *position only*, or *size only*. The finding (paraphrased): across four networks, the vertical position in the image wins over the apparent size of known obstacles.

- Position only tracks the true distance; size only does not.
- The result holds across four networks.
- Vertical position needs no object model and generalizes to objects absent from the training set.

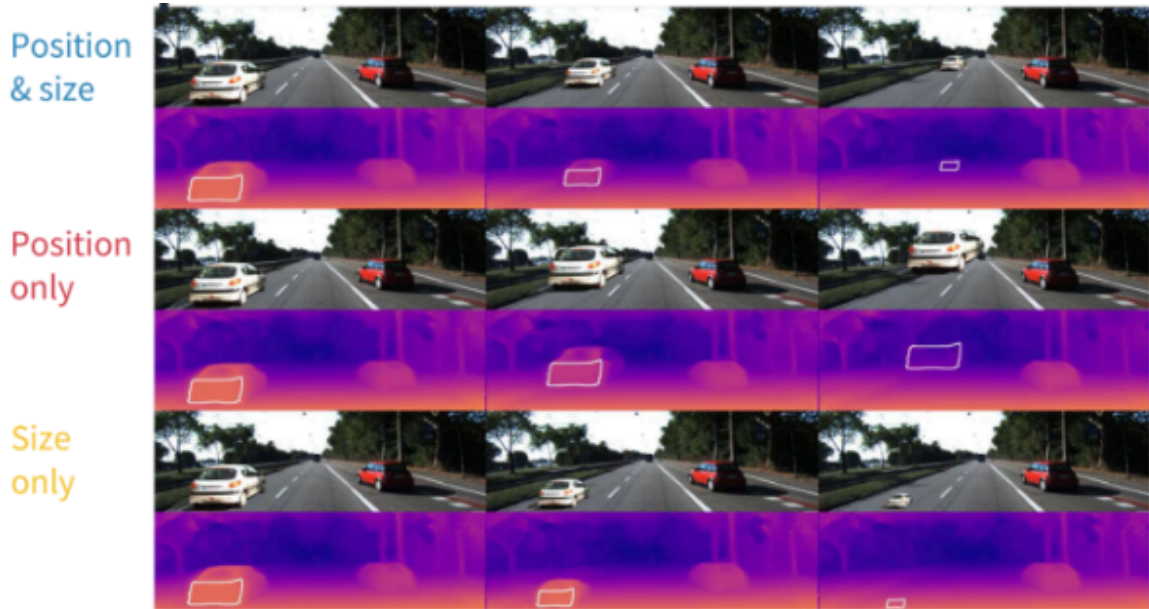


Figure 23: grid of pasted-car examples (position & size / position only / size only) with predicted depth maps

Intuition

The networks behave like the vertical-position cue of Proposition 3.7.6, not like Proposition 3.7.4. Sensible: vertical position works for *anything* touching the ground, whereas size needs a size prior per object class. But it also means a car pasted “floating” higher in the frame is judged farther, even if it appears huge.

3.9.2 Pitch and Roll (*slides 38*)

A pitch change is simulated with a vertical crop and a roll with a rotated crop; both are recovered from the predicted depth. Findings: the networks **only partially** track camera orientation, and small pitch changes measurably disturb the estimated distance to obstacles.

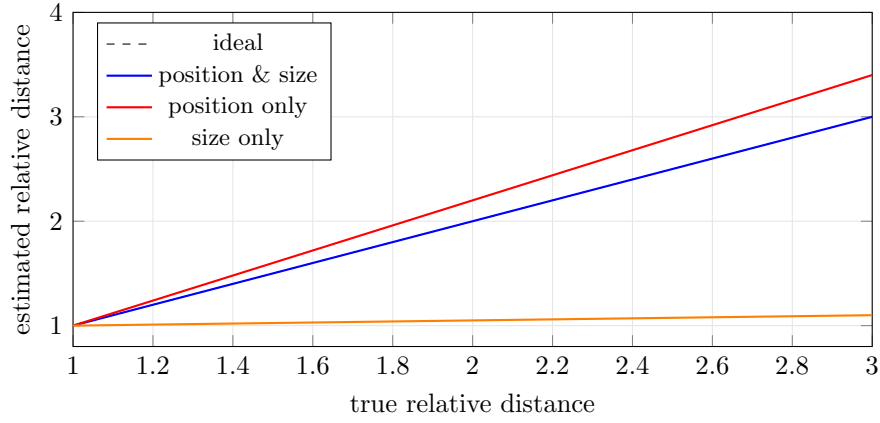


Figure 24: **Qualitative schematic** after the plot on slide 37, not measured data: positional cues track the ideal line (with some overestimate and larger spread), size-only stays almost flat.

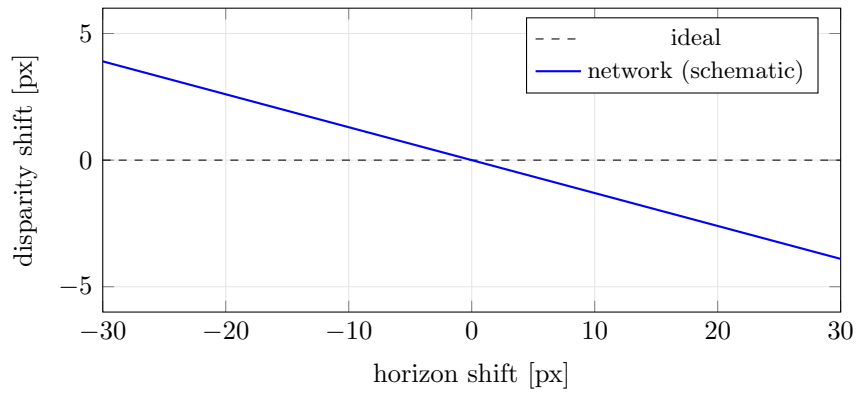


Figure 25: **Qualitative schematic** after the plot on slide 38, not measured data: an ideal system would ignore the crop (zero shift), the networks' output drifts roughly linearly and opposite to the horizon shift.

Intuition

A crop does *not* change an object’s distance to the horizon, $y_s - y_h$, so by Proposition 3.7.6 the true depth cannot change. But it does change the object’s *absolute* vertical position, and the predictions change with it. The network reads vertical image position, rather than distance to the horizon – exactly the shortcut warned about in §3.6, where the horizon depends on pitch and roll only.

3.9.3 Dependence on the Training Camera (*slides 39*)

A third probe: what happens when only the *camera* changes. A network trained at one focal length predicts the wrong depth at another, with the scene held identical. The apparent-size relation (Proposition 3.7.4) contains f , and the network has absorbed the training value as a constant. Supplying the intrinsics to the network as extra input channels removes most of the error. The slide’s summary: the prior is fitted to the training *camera* as much as to the training *scenes*.

Proposition 3.9.1 (Idealized focal-length bias – worked consequence, not on the slide). Suppose a network has absorbed $z = f_0 H / h$ with the training focal length f_0 . At test focal length f the image height is $h' = f H / z$, so the predicted depth is

$$\hat{z} = \frac{f_0 H}{h'} = \frac{f_0}{f} z.$$

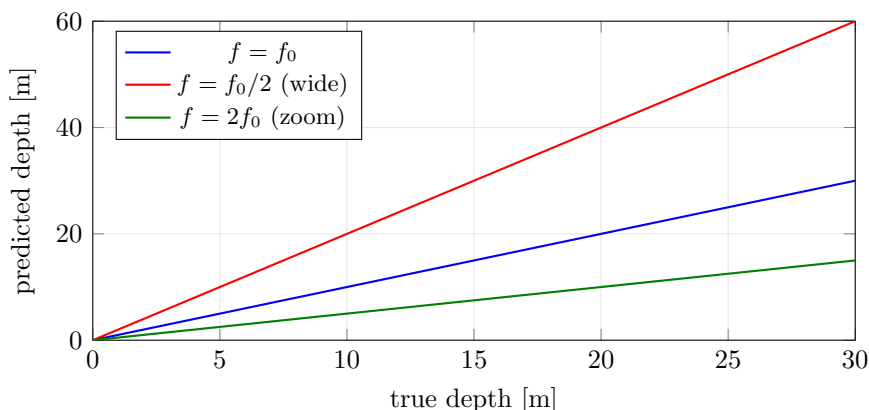


Figure 26: Original figure (not on the slides): predicted depth under the idealization of Proposition 3.9.1.

Intuition

Zooming in makes everything look bigger, and a network that silently assumes the training focal length reads “bigger” as “closer”. This is a scale error, so it is precisely the kind of ambiguity metric prediction must resolve – giving the network the intrinsics removes the hidden constant.

Next: Self-Supervision and a Generative Prior (*slides 40*)

Part I closes owing two debts: the prior was fitted to labels that a sensor had to measure, and mixing those labels cost two free parameters. Part II removes the labels, then pays back the two numbers.

Part I in One Slide (*slides 1–2*)

Part I obtained depth in two ways and closed *owing two debts*:

- **Time.** A laser measures range directly: metric and exact where it lands, but sparse, and it produces the labels everything else trains on.
- **A prior.** A network fitted to those labels is dense and cheap, but it *predicts* rather than recovers depth. The prior was fitted to labels that a sensor had to measure.

Mixing labels of different origin moved the output down a table, at a cost of free parameters.

Definition 3.9.2 (Output types and their free parameters). A depth-like output is defined only up to a transformation, and each transformation leaves free parameters that something else must later fix.

output	free parameters	aligned by
metric depth	0	nothing
scale-invariant depth	1	one scale s
affine-invariant disparity or depth	2	scale and shift (s, t)
ordinal	any monotone map	nothing

Intuition

Each free parameter is a *number a sensor must still supply*. If a network says “this pixel is twice as far as that one”, it has told you shape but not meters. Example: the truth is $D = 2.0 \hat{D} + 0.3$ m. A metric network outputs D itself; an affine-invariant one outputs $\hat{D}$ and you need the pair (2.0, 0.3) to recover meters. The whole lecture is about who supplies those numbers: the baseline of a stereo rig, a large generative prior, or a handful of LiDAR returns.

3.10 Supervision without Ground Truth (*slides 3–6*)

This is section 3.9 of the lecture. Its thesis: *a stereo pair supplies its own supervision, provided the warp connecting the two images is differentiable*.

3.10.1 View Synthesis and the Backward Warp (*slides 4*)

Training uses stereo pairs $\{(\mathbf{I}_L, \mathbf{I}_R)\}$ only; there is no depth anywhere in the training set. The objective is to reproduce one image of the pair from the other. Depth is the only internal quantity that makes this task solvable, so if the network solves it, depth is what it must have learned. This holds only where a pixel is **textured** and visible in **both** views; elsewhere the objective says nothing and a hand-written prior fills the gap.

Definition 3.10.1 (Backward warp). Given a disparity map $d(u, v)$ defined on the *target* image, the backward warp synthesizes the target from the source by

$$\tilde{\mathbf{I}}_L(u, v) = \mathbf{I}_R(u - d(u, v), v). \quad (3.10.1)$$

Every *target* pixel is mapped to exactly one *source* location.

Proposition 3.10.2 (Why the warp must be backward). *Forward mapping (pushing each source pixel to a target location) lets several sources land on one target pixel and leaves other target pixels empty: collisions and holes. Backward mapping assigns each target pixel exactly one value, which makes the warp a differentiable function of d rather than a scatter operation.*

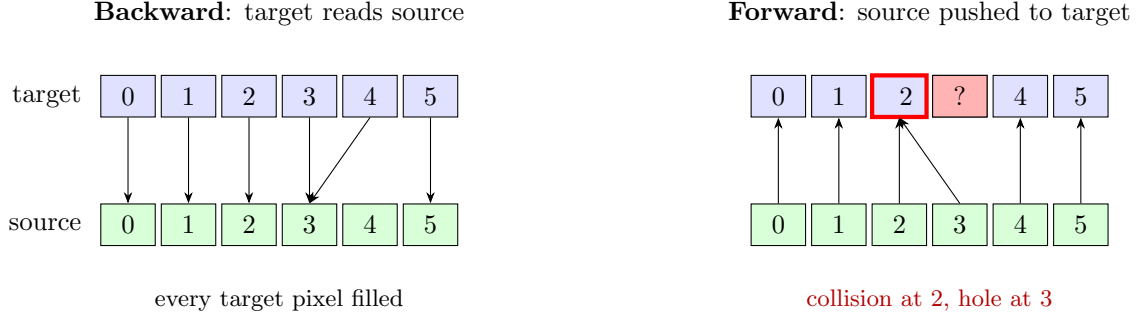


Figure 27: Original 1D illustration of backward versus forward mapping (slide 4 shows the same effect on small coloured grids).

Intuition

Think of filling in a form versus mailing letters. Backward warping is the form: each target pixel asks “*where in the other image should I look?*” and gets exactly one answer. Forward warping is the post office: every source pixel is mailed to an address; some mailboxes get two letters (collision), others none (hole), and “two letters or none” has no useful gradient. Note the sign convention: slide 4 writes $u - d$ for the left image as target, slide 5 writes $+d$ for a target/source pair set up the other way round; only the consistent use matters.

3.10.2 Image Reconstruction as Supervision (*slides 5*)

The pipeline of slide 5 has three steps: (1) a deep CNN predicts an inverse depth from the left image; (2) the right image is warped with it into a synthetic left image; (3) the reconstruction error against the real left image is the loss.

Definition 3.10.3 (Disparity and photometric loss). Under a rectified stereo rig with focal length f and baseline b , the predicted inverse depth is the disparity

$$d(\mathbf{p}) = \frac{f b}{z(\mathbf{p})}. \quad (3.10.2)$$

The self-supervised loss compares the warped and real image,

$$\mathcal{L}_{\text{photo}} = \sum_{\mathbf{p}} \|\tilde{\mathbf{I}}(\mathbf{p}) - \mathbf{I}_L(\mathbf{p})\|, \quad \tilde{\mathbf{I}}(\mathbf{p}) = \mathbf{I}_R(\mathbf{p} + d(\mathbf{p})). \quad (3.10.3)$$

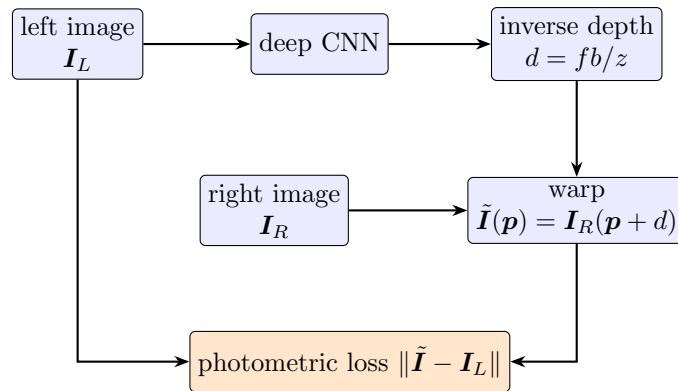


Figure 28: Self-supervised stereo training (redrawn from slide 5). No depth label appears anywhere.

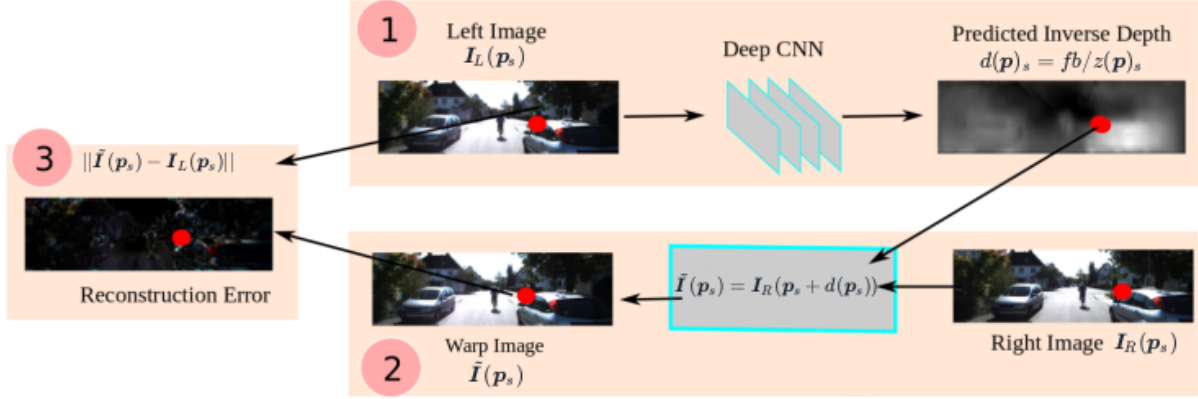


Figure 29: the KITTI street scene: left image, predicted inverse-depth map, warped image and reconstruction-error map (steps 1–3, with the red marker point)

Intuition

Depth is the *only* unknown the network can vary to make the warped image match the real one, so a network that solves the task has learned depth. Worked numbers (illustrative): with $f = 700$ px and $b = 0.5$ m, a car at $z = 10$ m has $d = 35$ px, whereas at $z = 70$ m it has only $d = 5$ px. Disparity is *inverse* depth: distant objects move the image little, so the training signal is weak at range.

3.10.3 Where the Loss Is Blind (*slides 6*)

Proposition 3.10.4 (Blind spots of the photometric loss). (a) Textureless regions. Writing the residual as $r = I_R(u - d) - I_L(u)$ and the squared loss as $\frac{1}{2}r^2$,

$$\frac{\partial \mathcal{L}}{\partial d} = -r \cdot I'_R(u - d), \quad (3.10.4)$$

which vanishes wherever $I(u_0 + 1) \approx I(u_0)$ (flat image, zero derivative): such pixels receive no training signal at all.

(b) Occlusions. A pixel visible to one camera only has no match in the other view, so the residual there is meaningless.

Both gaps are filled by hand-written priors: a smoothness term, left-right consistency, a minimum over the two reprojections, and masking of pixels that do not move between frames.

Proposition 3.10.5 (The baseline decides what the trained model is worth). Trained on a stereo pair, the model inherits the known baseline b and is metric for that rig. Trained on video, the baseline is unknown, so the output is shape up to one global scale and every image needs its own alignment.

Intuition

The photometric loss is a detective that only works with clues. Flat white wall: no clue, no gradient, so the smoothness prior guesses. Something hidden from one eye: no partner to compare with. The baseline is the ruler: with stereo the ruler length b is known, so meters come free; with a monocular video the ruler length is not known, so you get a beautiful shape

in unknown units (one free parameter, as in the table of Section 1).

3.11 Diffusion, From Scratch (*slides 7–22*)

This is section 3.10. Its thesis: *a model trained to remove noise is a prior: it points toward the data from anywhere in the space, and it can be sampled, conditioned or steered.* (Adapted on the slides from MIT 6.S183, Lecture 1, by D. Pfrommer.)

3.11.1 Supervised Learning (*slides 8*)

Supervised learning fits **one output per input**. At a depth edge that output is the average of the near surface and the far one: a distance that belongs to neither.

Proposition 3.11.1 (MSE regression returns the conditional mean). *Let Y be the target and $\mathbf{x}$ the input. Among all functions f , the minimizer of $\mathbb{E}\|f(\mathbf{x}) - Y\|^2$ is $f^*(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}]$.*

Proof. For fixed $\mathbf{x}$ with $m = \mathbb{E}[Y \mid \mathbf{x}]$, $\mathbb{E}[\|f - Y\|^2 \mid \mathbf{x}] = \|f - m\|^2 + \mathbb{E}[\|Y - m\|^2 \mid \mathbf{x}]$; the second term does not depend on f , and the first is minimized by $f = m$. $\square$

Intuition

At a depth edge half the training examples say 2 m (near surface) and half say 10 m (far surface). The best single answer under squared error is 6 m, which is the depth of *nothing*. That is the blurred, flying-pixel edge of regression networks.

3.11.2 Generative Learning (*slides 9*)

A generative model fits a **distribution** over outputs, not one value. “The band is what a diffusion model represents.”

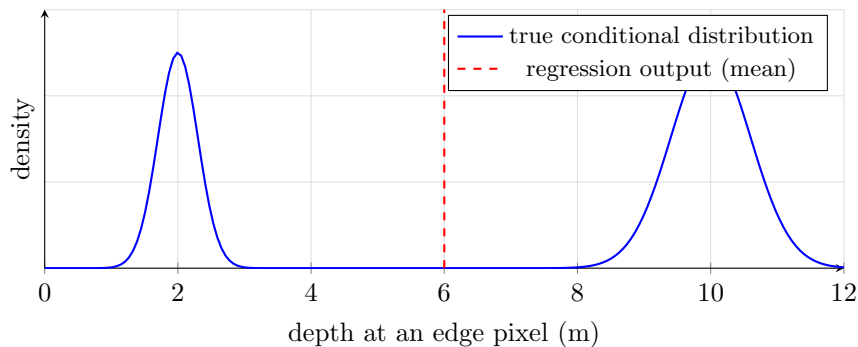


Figure 30: Original toy figure: a bimodal conditional distribution at a depth edge. The regressor answers the mean, which lies in a gap where the density is zero; a generative model can return a sample from one of the two modes.

3.11.3 Many Outputs for One Input – Text to Image (*slides 10*)

One conditioning input has many valid completions. A regressor averages them; a diffusion model picks one.

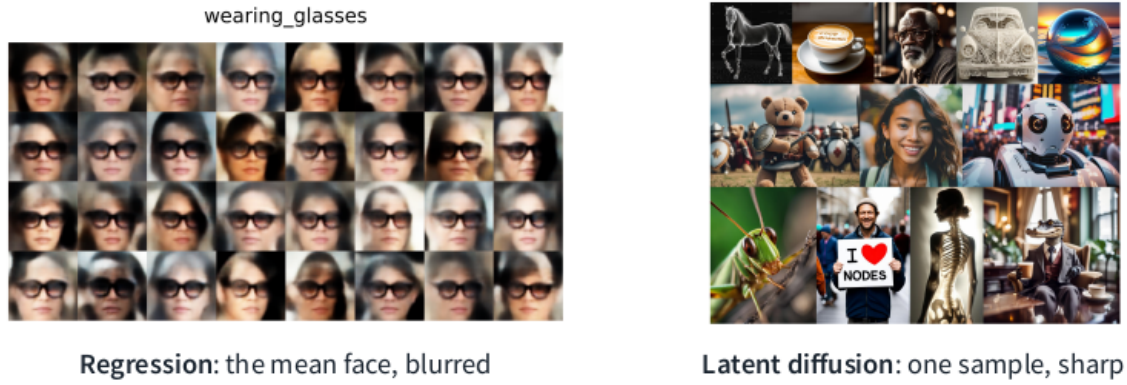


Figure 31: left: grid of “wearing_glasses” faces from regression (the mean face, blurred); right: grid of latent-diffusion samples (one sample, sharp)

Intuition

Blur is not a training failure. It is the correct answer to the wrong question: the conditional mean of a set of sharp images is not a sharp image. The same averaging is what flattens a depth edge.

3.11.4 The Forward and Reverse Process (*slides 11*)

The **forward process** adds noise to data one step at a time until nothing of the data is left. It is fixed and has no parameters. It is inspired by Brownian motion (many small independent displacements), and after enough steps the sample is indistinguishable from Gaussian noise. A diffusion model learns the **reverse process**, which recovers data from noise; that reverse process *is the prior*.

Definition 3.11.2 (Forward and reverse processes). The forward chain $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ maps $\mathbf{x}_0$ (data) to $\mathbf{x}_T$ (noise) and is fixed; the model $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ learns the way back.

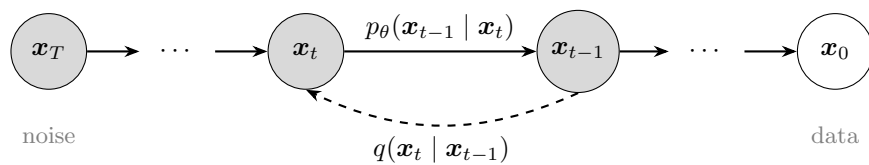


Figure 32: Forward (dashed, fixed) and reverse (solid, learned) chains, redrawn from slide 11.

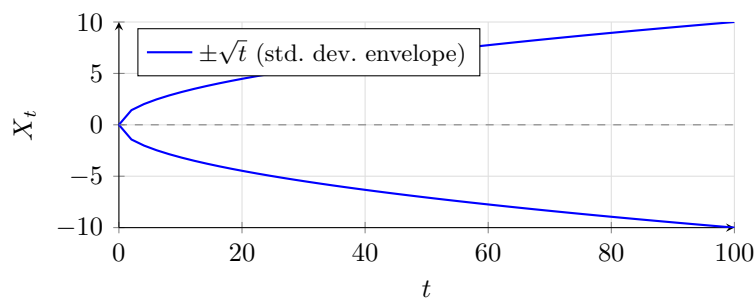


Figure 33: Original figure: the spread of a random walk grows like $\sqrt{t}$, the mechanism that buries the data in noise.

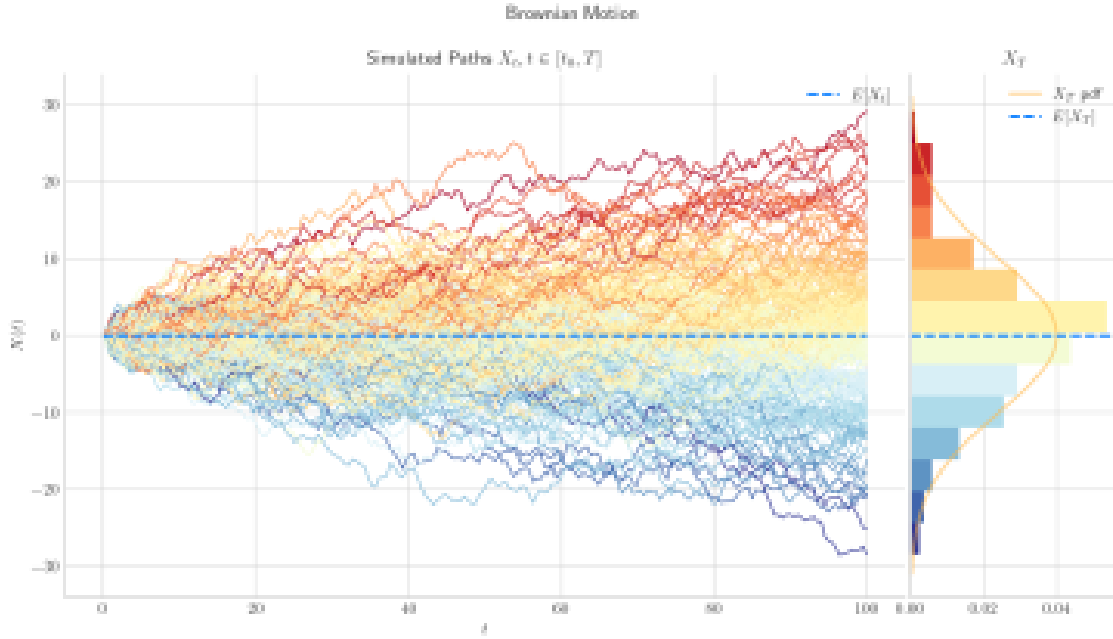


Figure 34: Brownian-motion simulated paths (coloured bundle of trajectories) with the histogram of X_T on the right, and the two small face images (noisy, clean) below the chain

3.11.5 Training by Noise Prediction (*slides 12*)

A denoising diffusion model estimates the noise vector ϵ from a noise level $\sigma > 0$ and a noisy input $\mathbf{x}_\sigma \approx \mathbf{x}_0 + \sigma\epsilon$, where $\mathbf{x}_0$ lies on the data manifold $\mathcal{K} \subset \mathbb{R}^n$.

Definition 3.11.3 (Denoiser and training loss). A denoiser $\epsilon_\theta : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}^n$ is learned by minimizing

$$L_{\text{simple}}(\theta) := \mathbb{E}_{\mathbf{x}_0, \sigma, \epsilon} \left[\left\| \epsilon_\theta(\mathbf{x}_0 + \sigma\epsilon, \sigma) - \epsilon \right\|^2 \right], \quad (3.11.1)$$

with $\mathbf{x}_0$ sampled from the training data, σ from a training noise schedule, and $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Intuition

The recipe is three lines: take a clean example, add noise of a random strength you remember, and ask the network “what noise did I add?”. Subtracting the guessed noise gives the guessed clean point, $\hat{\mathbf{x}}_0 = \mathbf{x}_\sigma - \sigma\epsilon_\theta$. No labels beyond the data itself are needed.

3.11.6 The Spiral Toy Model (*slides 13*)

Everything that follows is shown on a 2D data set, where the prior is visible: *it is where the points are*. The data are points of a 2D spiral $\mathbf{x}_0$. The network is a fully connected MLP fed with $\mathbf{x}_0 + \sigma\epsilon$ and a 2D noise-level embedding of σ , and it predicts the noise.

```
def get_sigma_embeds(sigma):
    sigma = sigma.unsqueeze(1)
    return torch.cat([torch.sin(torch.log(sigma)/2),
                      torch.cos(torch.log(sigma)/2)], dim=1)

class TimeInputMLP(nn.Module):
    def __init__(self, dim, hidden_dims):
        super().__init__()
```

```

layers = []
for in_dim, out_dim in pairwise((dim + 2,) + hidden_dims):
    layers.extend([nn.Linear(in_dim, out_dim), nn.GELU()])
layers.append(nn.Linear(hidden_dims[-1], dim))
self.net = nn.Sequential(*layers)
self.input_dims = (dim,)

def forward(self, x, sigma):
    sigma_embeds = get_sigma_embeds(sigma)
    nn_input = torch.cat([x, sigma_embeds], dim=1)
    return self.net(nn_input)

model = TimeInputMLP(dim=2, hidden_dims=(16,128,128,128,128,16))

```

Intuition

The extra two inputs ($\sin, \cos$) of $\log \sigma/2$ tell the network *how noisy* its input is. This matters because the right answer depends on σ : at small σ the noise is a small nudge off the nearest data point, at large σ the input is almost pure noise. Note that the first layer has width $\dim + 2 = 4$: two coordinates plus the two embedding numbers.

3.11.7 Training the Toy Model (*slides 14*)

```

def training_loop(loader, model, schedule, epochs=10000):
    optimizer = torch.optim.Adam(model.parameters())
    for _ in range(epochs):
        for x0 in loader:
            optimizer.zero_grad()
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = model(x0 + sigma * eps, sigma)
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()

```

The learned denoiser can be drawn as a **vector field** by plotting $\mathbf{x} - \sigma \epsilon_\theta(\mathbf{x}, \sigma)$. At small noise the arrows point at the nearest data. At large noise they point at the middle of the whole data set. The slide's training curve falls from about 0.70 to about 0.41 over 15 000 epochs.

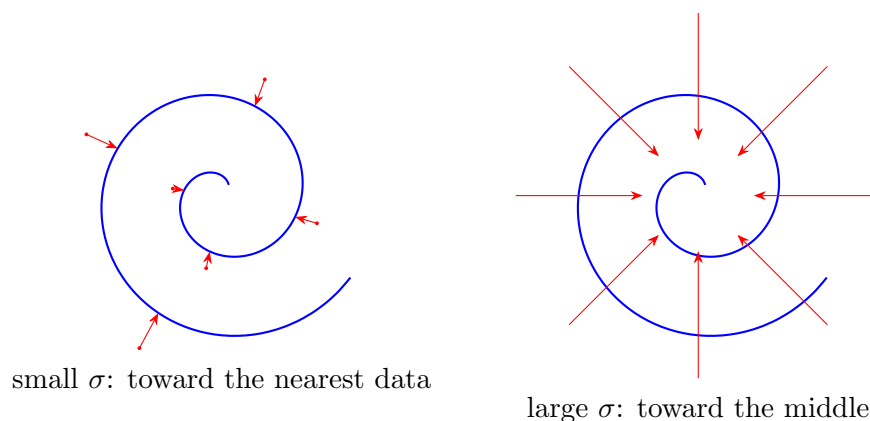


Figure 35: Original sketch of the denoiser field on the spiral (compare slide 14).

Remark 3.11.4 (Why the loss does not reach zero). The curve plateaus near 0.41, not 0. This is expected: even the best possible denoiser cannot recover ϵ exactly, because many $(\mathbf{x}_0, \epsilon)$ pairs produce the same $\mathbf{x}_\sigma$; the floor equals the average posterior variance of ϵ (Appendix A,

item A.2).

3.11.8 The Ideal Denoiser (slides 15)

The **ideal denoiser** minimizes the training loss; it is a function of the data set $\mathcal{K}$ and the noise level, and nothing else:

$$\epsilon^* := \arg \min_{\epsilon_\theta} \mathbb{E}_{\mathbf{x}_0, \sigma, \epsilon} \|\epsilon_\theta(\mathbf{x}_0 + \sigma\epsilon, \sigma) - \epsilon\|^2. \quad (3.11.2)$$

Theorem 3.11.5 (Ideal denoiser for a finite data set). *For a finite $\mathcal{K}$ (data uniformly distributed on $\mathcal{K}$, Gaussian noise),*

$$\epsilon^*(\mathbf{x}_\sigma, \sigma) = \frac{\sum_{\mathbf{x}_0 \in \mathcal{K}} (\mathbf{x}_\sigma - \mathbf{x}_0) \exp(-\|\mathbf{x}_\sigma - \mathbf{x}_0\|^2/2\sigma^2)}{\sigma \sum_{\mathbf{x}_0 \in \mathcal{K}} \exp(-\|\mathbf{x}_\sigma - \mathbf{x}_0\|^2/2\sigma^2)}. \quad (3.11.3)$$

It is a weighted average of all data points: nearer points count more.

Proof. By the conditional-mean proposition, $\epsilon^*(\mathbf{x}_\sigma, \sigma) = \mathbb{E}[\epsilon \mid \mathbf{x}_\sigma]$ with $\epsilon = (\mathbf{x}_\sigma - \mathbf{x}_0)/\sigma$. By Bayes, $p(\mathbf{x}_0 \mid \mathbf{x}_\sigma) \propto \exp(-\|\mathbf{x}_\sigma - \mathbf{x}_0\|^2/2\sigma^2)$ on $\mathcal{K}$. Taking the expectation of $(\mathbf{x}_\sigma - \mathbf{x}_0)/\sigma$ under these weights gives the formula. $\square$

Intuition

Worked 1D example with $\mathcal{K} = \{-1, +1\}$, $\mathbf{x}_\sigma = 0.2$. For $\sigma = 0.5$ the weights are $e^{-1.44/0.5} \approx 0.056$ (for -1) and $e^{-0.64/0.5} \approx 0.278$ (for $+1$), giving $\epsilon^* \approx -0.93$ and clean estimate $\hat{\mathbf{x}}_0 = 0.2 - 0.5 \cdot (-0.93) \approx 0.66$: pulled toward the nearer point $+1$. For a large $\sigma = 5$ the two weights are almost equal (0.972 vs 0.987), $\epsilon^* \approx 0.04$ and $\hat{\mathbf{x}}_0 \approx 0.01$: the arrow points at the middle of the data set, exactly as in the slide's three panels ($\sigma = 0.01, 0.1, 0.5$).

3.11.9 The Smoothed Distance Function (slides 16)

The distance to a set, $\text{dist}_{\mathcal{K}}(\mathbf{x}) := \inf\{\|\mathbf{x} - \mathbf{x}_0\| : \mathbf{x}_0 \in \mathcal{K}\}$, is not differentiable everywhere, so its gradient is hard to learn. Smoothing it by σ fixes that.

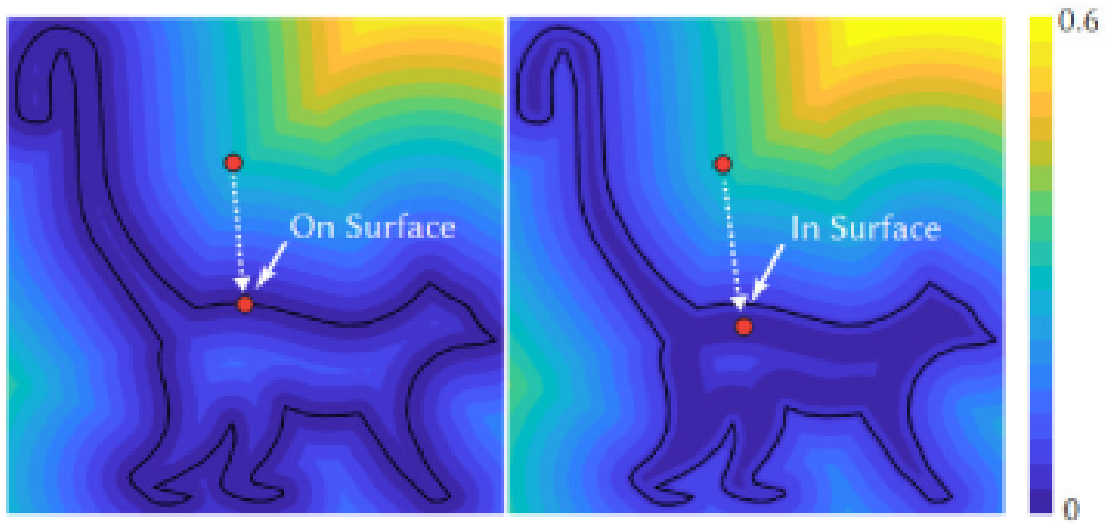
Definition 3.11.6 (Smoothed distance).

$$D_\sigma(\mathbf{x}) := -\sigma^2 \log \left(\sum_{\mathbf{x}_0 \in \mathcal{K}} \exp(-\|\mathbf{x}_0 - \mathbf{x}\|^2/2\sigma^2) \right). \quad (3.11.4)$$

(The slide denotes the smoothed squared distance by $\text{dist}_{\mathcal{K}}^2(\mathbf{x}, \sigma)$ and writes the identity below with a factor $\frac{1}{2}$; with the definition as printed, $\frac{1}{2}\text{dist}_{\mathcal{K}}^2(\mathbf{x}, \sigma)$ corresponds to D_σ .)

Proposition 3.11.7 (Denoiser = gradient of the smoothed distance). $\frac{1}{2}\nabla_{\mathbf{x}} \text{dist}_{\mathcal{K}}^2(\mathbf{x}, \sigma) = \nabla_{\mathbf{x}} D_\sigma(\mathbf{x}) = \sigma \epsilon^*(\mathbf{x}, \sigma)$, and $\min_{\mathbf{x}_0} \frac{1}{2}\|\mathbf{x} - \mathbf{x}_0\|^2 - \sigma^2 \log |\mathcal{K}| \leq D_\sigma(\mathbf{x}) \leq \min_{\mathbf{x}_0} \frac{1}{2}\|\mathbf{x} - \mathbf{x}_0\|^2$.

Proof. Differentiating, $\nabla D_\sigma = -\sigma^2 \cdot \frac{\sum e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2/2\sigma^2} \cdot (-2(\mathbf{x} - \mathbf{x}_0)/\sigma^2)}{\sum e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2/2\sigma^2}} = \frac{\sum (\mathbf{x} - \mathbf{x}_0) e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2/2\sigma^2}}{\sum e^{-\|\mathbf{x}_0 - \mathbf{x}\|^2/2\sigma^2}} = \sigma \epsilon^*$ by Theorem 3.11.5. The bounds follow from $e^{-a_{\min}/\sigma^2} \leq \sum_i e^{-a_i/\sigma^2} \leq |\mathcal{K}| e^{-a_{\min}/\sigma^2}$ with $a_i = \frac{1}{2}\|\mathbf{x} - \mathbf{x}_{0,i}\|^2$. $\square$



A smoothed distance function has continuous gradients

Figure 36: the two coloured distance-field images of the cat silhouette (“On Surface” and “In Surface” with the dotted arrows), with the colour bar 0–0.6

Intuition

min has kinks where two data points are equally near; a *soft*-min has none, and its slope is the vector from the weighted mean of the data to $\mathbf{x}$. So the denoiser is a compass that is defined *everywhere* and always points toward the data: that is what makes it a prior. Equivalently $\hat{\mathbf{x}}_0 = \mathbf{x}_\sigma - \sigma \epsilon^*$ is simply the weighted mean of the data points.

3.11.10 Sampling From a Diffusion Model – Why iterative? (*slides 17*)

Given a noisy $\mathbf{x}_\sigma$ and its noise level, the denoiser estimates the clean point directly,

$$\hat{\mathbf{x}}_0(\mathbf{x}_\sigma, \sigma) := \mathbf{x}_\sigma - \sigma \epsilon_\theta(\mathbf{x}_\sigma, \sigma). \quad (3.11.5)$$

A sampler walks a decreasing sequence of noise levels, taking one such estimate at each step.

DDIM is the update

$$\mathbf{x}_{t-1} = \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \epsilon_\theta(\mathbf{x}_t, \sigma_t). \quad (3.11.6)$$

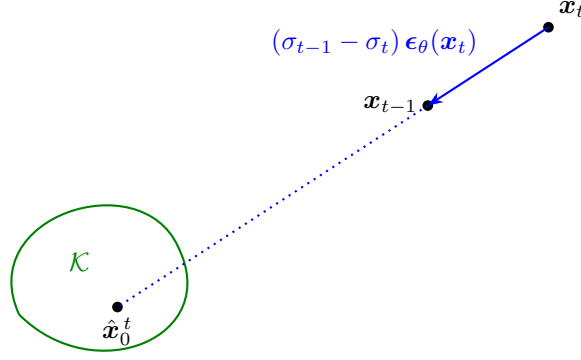


Figure 37: DDIM step (redrawn from slide 17): a straight line toward the current estimate $\hat{x}_0^t$ of the data.

Intuition

One step goes only *part* of the way to the current best guess, because the guess is poor while the input is very noisy; after moving, you re-evaluate the compass. The step is a straight line toward the estimate. This is the equation that depth completion reaches inside and steers (Section 3.14). Algebraically the step re-noises the clean estimate to the lower level: $\mathbf{x}_{t-1} = \hat{\mathbf{x}}_0^t + \sigma_{t-1} \epsilon_\theta$ (Appendix A).

3.11.11 The DDIM Sampler (slides 18)

Algorithm 3.11.8 (DDIM sampler). for $t = N, \dots, 1$: $\mathbf{x}_{t-1} \leftarrow \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \epsilon_\theta(\mathbf{x}_t, \sigma_t)$; return $\mathbf{x}_0$.
Start from $\mathbf{x}_N = \sigma_N \cdot \mathcal{N}(\mathbf{0}, \mathbf{I})$.

```
@torch.no_grad()
def samples_ddim(model, sigmas, batchsize=1):
    xt = model.rand_input(batchsize) * sigmas[0]
    for sig, sig_prev in pairwise(sigmas):
        xt = xt - (sig - sig_prev) * model.predict_eps(xt, sig)
    return xt
```

On the toy model: *fifty steps, no retraining, and the trajectory follows the field.*

3.11.12 Probabilistic Sampling (slides 19)

DDIM is deterministic: the same starting noise always gives the same sample. Adding a noise term back makes the sampler stochastic:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \epsilon_\theta(\mathbf{x}_t, \sigma_t) + \mu \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (3.11.7)$$

(The slide writes $\sigma_{t'}$ for the next, lower level.) With $\mu = 0$ this is DDIM again; as μ grows the trajectories spread, and the same input gives different samples. The slide shows $\mu = 0.001, 0.01, 0.1$.

Intuition

μ is a “temperature”: $\mu = 0$ rolls a ball down a fixed groove, $\mu > 0$ shakes the table on the way. This is the reason different seeds give different depth maps in Marigold (Section 3.12).

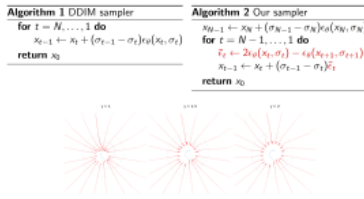
3.11.13 Three Uses of a Prior (*slides 20*)

A prior that exists as a **separate object** can be used in three ways, each a later section of the lecture.

use	meaning	section
Sample	draw from the distribution the model represents	3.10
Condition	train it with a side input $\mathbf{c}$, so it samples from $p(\mathbf{x} \mathbf{c})$	3.11
Guide	modify the state inside the sampling loop at test time, with no retraining	3.13

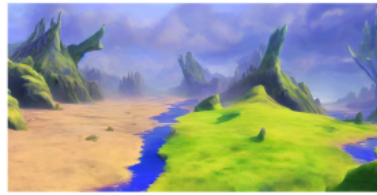
The third is the one that turns a depth prior into a metric depth map.

1. Sample from it



Draw from the distribution the model represents (what we just saw).

2. Condition it



Train it with a side input $\mathbf{c}$, so it samples from $p(\mathbf{x} | \mathbf{c})$.

3. Guide it



Modify the state inside the sampling loop at test time, with no retraining.

Figure 38: the three panels: sampler comparison plots (Algorithm 1 vs Algorithm 2), the generated landscape (conditioning), and the super-resolution / inpainting / colorization examples (guidance)

3.11.14 The Autoencoder and Latent Diffusion (*slides 21*)

Images are big, so an autoencoder compresses them first: an encoder $\mathcal{E}$ maps $\mathbf{x}$ to a latent $\mathbf{z}$, and a decoder $\mathcal{D}$ maps back, $\mathbf{z} = \mathcal{E}(\mathbf{x})$, $\tilde{\mathbf{x}} = \mathcal{D}(\mathbf{z})$. The autoencoder is trained **first**, then **frozen**; the diffusion model never sees pixels and you decode once, at the end. Diffusion runs on $\mathbf{z}$ with the same loss:

$$L = \mathbb{E}_{\sigma, \mathbf{z}, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(\mathbf{z}_0 + \sigma \epsilon, \sigma) \right\|^2 \right]. \quad (3.11.8)$$

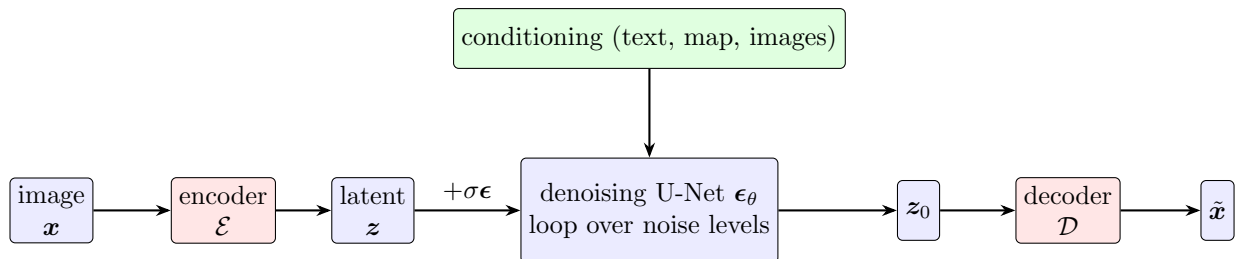


Figure 39: Latent diffusion in outline (original, inspired by slide 21). Pink boxes are frozen autoencoder parts.

Intuition

Worked numbers: $768 \times 768 \times 3 \rightarrow 96 \times 96 \times 4$, i.e. 1 769 472 numbers become 36 864: eight times smaller per side, four channels, **48 times fewer numbers**. Each of the 50 denoising steps is therefore 48 times cheaper than it would be in pixel space.

3.11.15 Conditioning and Guidance (*slides 22*)

Conditioning is a side input the network was *trained with*; the model responds to it because it was trained to. **Guidance** modifies the sampling trajectory *at test time*, using a criterion the network was never trained on. Sampling is a loop, and its state can be modified between steps.

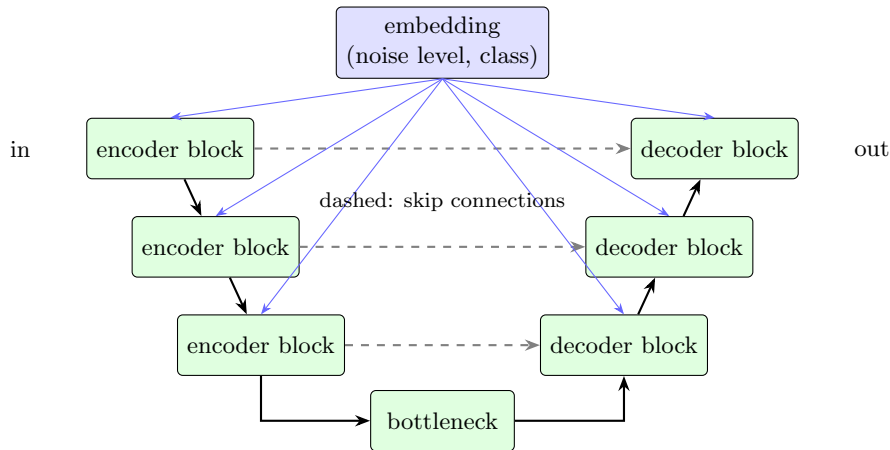


Figure 40: Simplified U-Net redrawn from slide 22. Each block is GroupNorm–SiLU–Conv 3×3 with the embedding injected through a Linear layer (scale and shift); attention is optional.

3.12 Marigold (*slides 23–31*)

Section 3.11: a depth estimator from an image diffusion model. Stable Diffusion’s autoencoder is kept, only its denoiser is fine-tuned, and the output is a depth map rather than an image.

3.12.1 The Marigold Method (*slides 24*)

The argument: a monocular estimator’s knowledge of the visual world is **bounded by its training set**, while a text-to-image model trained on billions of captioned photographs has already seen it. Hence: if the cornerstone of monocular depth is a comprehensive representation of the visual world, one should be able to derive a depth estimator from a pretrained image diffusion model.

Definition 3.12.1 (Marigold). Take a pretrained text-to-image latent diffusion model, **freeze the VAE**, and fine-tune only the denoising U-Net so that what it denoises is a *depth* latent rather than an image latent. Properties from the slide: affine-invariant monocular depth; trained on **synthetic data only**; about 2.5 days on a single consumer GPU.

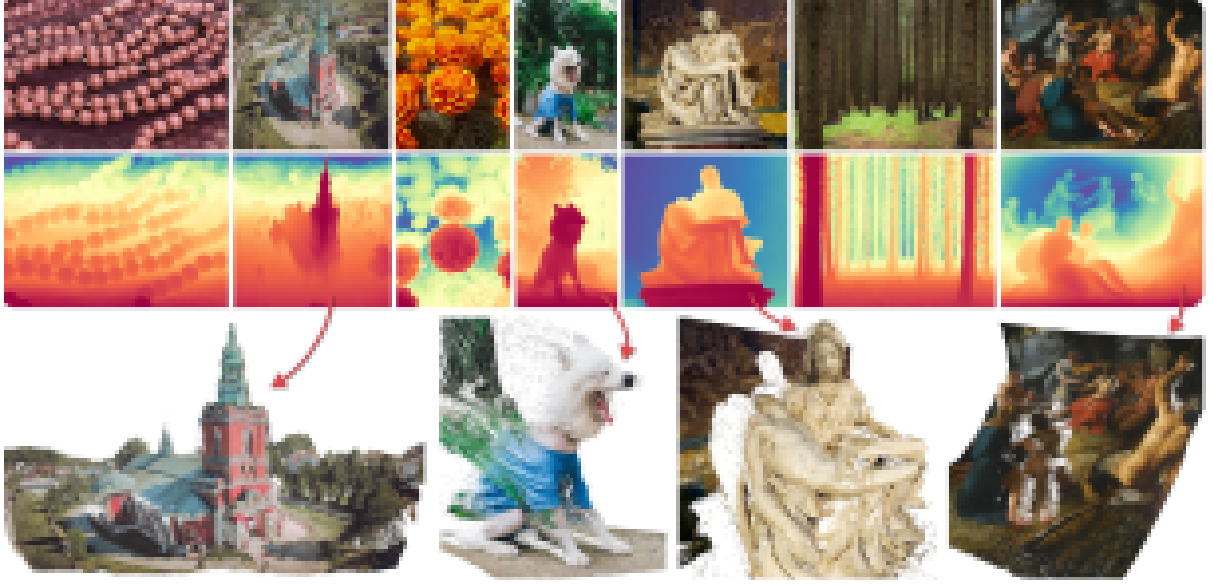


Figure 41: the strip of example results: photographs (beads, church, marigolds, dog, sculpture, forest, painting) with their depth maps and back-projected 3D views

3.12.2 From the Spiral to Marigold (*slides 25*)

Marigold is the spiral model with one extra input; nothing in the algorithm changes.

	toy model	Marigold
starts from	random weights	Stable Diffusion's weights
a data point	a 2D point	a depth latent
the denoiser	a small MLP	Stable Diffusion's U-Net
the condition	none	the image latent
trained on	spiral points	74K rendered image–depth pairs
the sampler	DDIM	the same DDIM
a sample is	a point on the spiral	a depth map the RGB allows

The first row is the whole bet: *the visual prior is already in those weights*, and fine-tuning moves what they denoise from images to depth. (Stable Diffusion indexes noise levels by a timestep t and scales its input; both are fixed functions of σ , so the process is the same.)

3.12.3 The Source of Affine Invariance (*slides 26*)

The normalization defines the output space: the affine-invariant *depth* row of the table in Part I, not the affine-invariant *disparity* row that MiDaS uses.

Definition 3.12.2 (Percentile normalization). With D_2 and D_{98} the 2nd and 98th percentiles of the depth map D , the training target is

$$\tilde{D} = \left(\frac{D - D_2}{D_{98} - D_2} - 0.5 \right) \times 2, \quad \text{and metric depth is recovered as} \quad \tilde{D}_{\text{metric}} = s\tilde{D} + t. \quad (3.12.1)$$

Proposition 3.12.3 (Normalization removes scale and shift). *If $D' = aD + b$ with $a > 0$, then $\tilde{D}' = \tilde{D}$.*

Proof. Percentiles commute with increasing affine maps: $D'_2 = aD_2 + b$, $D'_{98} = aD_{98} + b$. Hence $(D' - D'_2)/(D'_{98} - D'_2) = a(D - D_2)/(a(D_{98} - D_2))$, the same ratio. $\square$

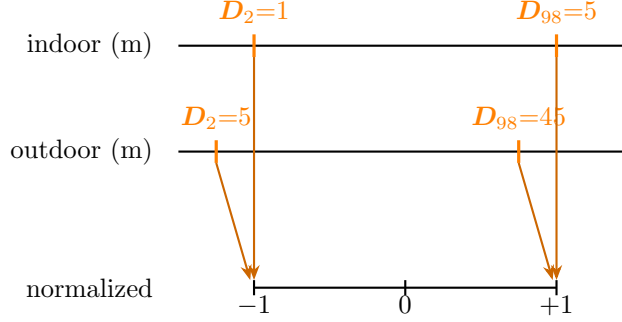


Figure 42: Original figure: an indoor map (few meters) and an outdoor map (tens of meters) land on the same $[-1, +1]$ range; each map is normalized by its *own* percentiles (slide 26). The numbers are illustrative.

Intuition

Every training map is stretched to fill $[-1, +1]$, so a room and a highway look identical to the network in terms of range. **The model never sees meters and never sees a common zero, so it cannot learn them; it is constrained to predict shape.** Example: in a room with $D_2 = 1$ m and $D_{98} = 5$ m, a pixel at 3 m maps to 0; on a street with $D_2 = 5$ m and $D_{98} = 45$ m, a pixel at 25 m also maps to 0.

3.12.4 Conditioning on the Image (*slides 27*)

The denoiser takes the noisy depth latent $z_\sigma^{(D)}$ and the image latent $z^{(I)}$:

$$\epsilon_\theta(z_\sigma^{(D)}, z^{(I)}, \sigma), \quad z_\sigma = \text{cat}(z_\sigma^{(D)}, z^{(I)}). \quad (3.12.2)$$

The depth map is replicated to three channels and encoded by the **frozen RGB autoencoder**, whose reconstruction error on depth is negligible. **Concatenation** suits a condition that is spatially aligned with the output, pixel for pixel; cross-attention is for conditions that are not.

Definition 3.12.4 (Four changes to the network). (1) Concatenate the image latent and the noisy depth latent along the channels; (2) double the U-Net’s input channels; (3) duplicate the first layer’s weights and **halve** them, so activation magnitudes do not inflate; (4) **disable text conditioning** (original Stable Diffusion).

Intuition

Why halve? Let the original first layer be Wz . The new layer sees $[z; z]$ and uses $[W/2, W/2]$, so its output is $Wz/2 + Wz/2 = Wz$: at the start of fine-tuning the pretrained network behaves as before. Without halving, the same input would produce $2Wz$, twice the activations the pretrained weights expect. Since the latent has 4 channels, the new input has 8. Concatenation works here because the depth map and the image are aligned pixel by pixel.

3.12.5 Training Marigold (*slides 28*)

```

def training_loop(loader, unet, vae, schedule, epochs):
    optimizer = torch.optim.Adam(unet.parameters())
    for _ in range(epochs):
        for image, depth in loader: # pairs
            optimizer.zero_grad()
            # start lock
            x0 = vae.encode(normalize(depth)) # a depth latent
            cond = vae.encode(image) # the condition
            # end lock
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = unet(x0 + sigma*eps, cond, sigma) # sees both
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()

```

The objective is the one from the spiral, unchanged. Only the U-Net trains; the VAE is frozen at both ends. Data: 74K rendered pairs, about 2.5 days on one consumer GPU.

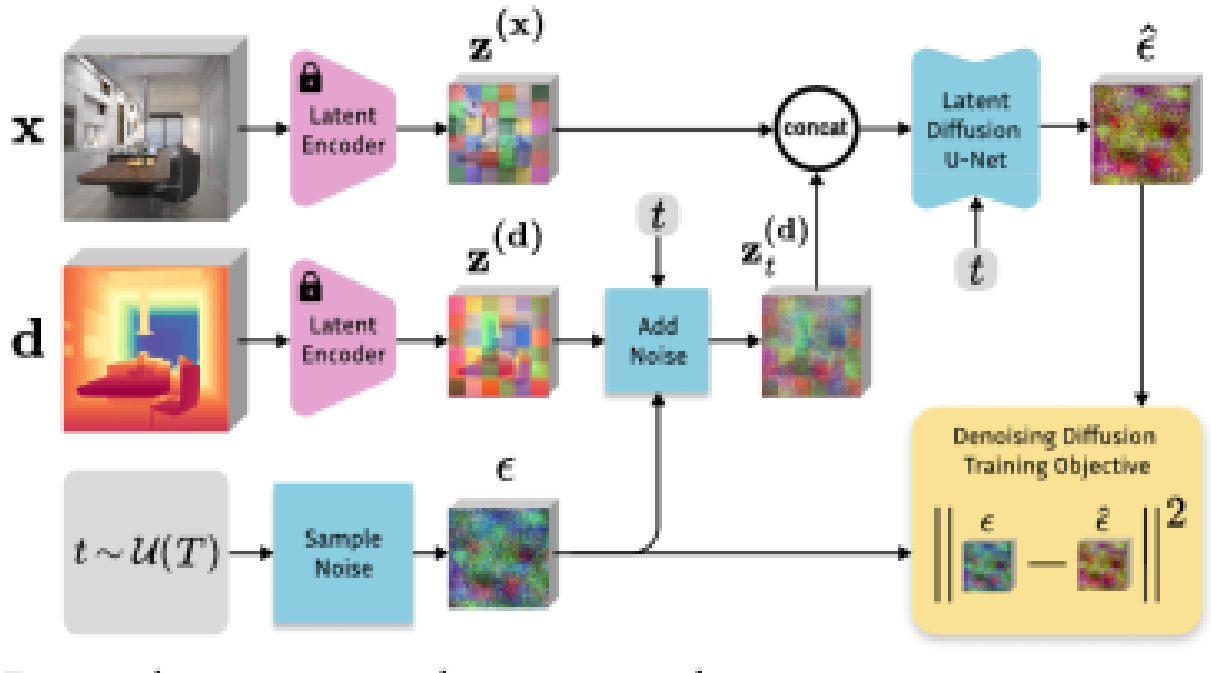


Figure 43: Marigold training (redrawn from slides 27–28). Pink: frozen. The noise ϵ is also fed to the loss.

3.12.6 Inference (slides 29)

Algorithm 3.12.5 (Marigold inference, one run). (1) Encode the image **once**; (2) initialize the depth latent $z_T^{(D)}$ as pure Gaussian noise; (3) run the denoising loop, concatenating the image latent at **every** step; (4) decode $z_0^{(D)}$ and average the three channels. From the paper: 50 DDIM steps.

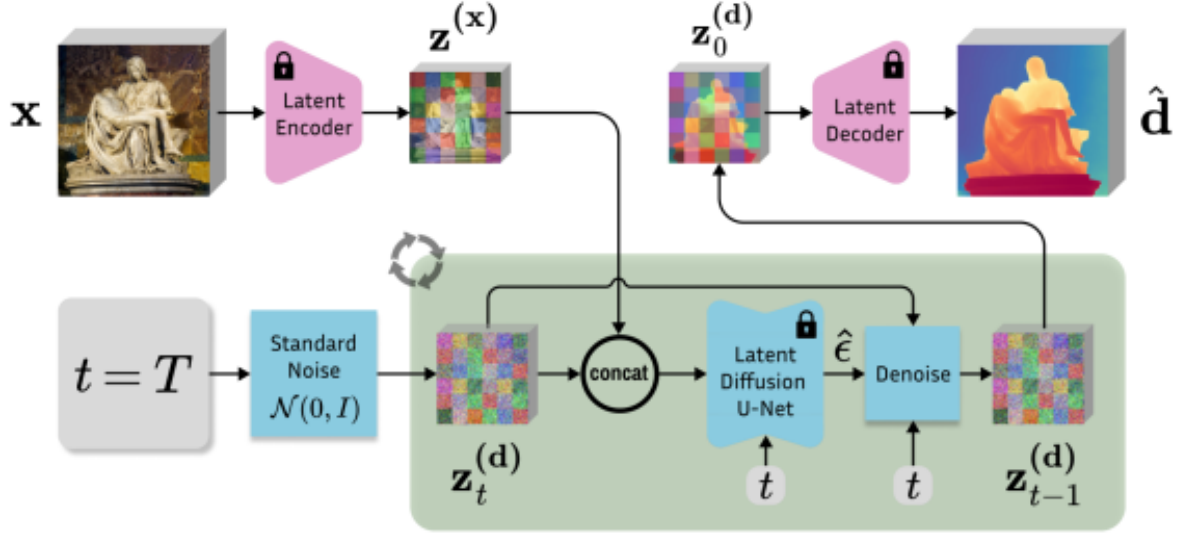


Figure 44: Marigold inference

3.12.7 Ensembling Over Samples (*slides 30*)

Probabilistic sampling is why different seeds give different depth maps; Marigold runs the loop N times and combines the results.

Algorithm 3.12.6 (Ensembling). (1) Each run is affine-invariant on its own scale, so first fit a scale and a shift per run so that the maps agree with each other (minimize the pairwise distance between the aligned maps, with a penalty that keeps the range on $[0, 1]$); (2) take the pixel-wise **median**. Default $N = 10$; **no ground truth is used**.

Intuition

Ten drafts of the same drawing, each in its own unknown units. First rescale and shift each draft to line up with the others (they only agree up to (s_i, t_i)), then let the median vote at each pixel, which ignores outlier drafts.

3.12.8 Limitations of Marigold (*slides 31*)

- **No metric scale.**
- **No zero:** not even a common origin between two images of the same room.
- **Slow:** 50 denoising steps $\times$ 10 ensemble members = 500 network evaluations per image.
- **It returns a sample** (non-deterministic).

3.13 Back to 3D (*slides 32–34*)

Section 3.12: back-projection turns a depth map into points and shows which of the unknown parameters distort the result.

3.13.1 Effect of Scale and Shift Errors (*slides 33*)

A depth map is a 2.5D surface: one depth per pixel, with nothing behind what is visible.

Definition 3.13.1 (Back-projection). A pixel with intrinsics-normalized ray $\mathbf{r} = \mathbf{K}^{-1}(u, v, 1)^\top$ and depth z back-projects to $\mathbf{p} = z \mathbf{r}$.

Proposition 3.13.2 (Affine depth error). *If $\hat{z} = sz + t$, the reconstructed point is $\hat{\mathbf{p}} = (s + t/z)\mathbf{p}$. In particular: the **scale** s multiplies the whole cloud (a similarity, so shape is preserved); the **shift** t drags every point a constant depth along its own ray, a different displacement in every direction (the shape can distort).*

Proof. $\hat{\mathbf{p}} = \hat{z} \mathbf{r} = (sz + t)\mathbf{r} = (s + t/z)z\mathbf{r}$. □

Corollary 3.13.3 (Affine error in disparity). *If disparity is $\hat{d} = sd + t$ then $\hat{z} = 1/\hat{d}$ and $\hat{\mathbf{p}} = \mathbf{p}/(s + tz)$: a projective map of space.*

Proof. $\hat{z} = \frac{1}{s/z + t} = \frac{z}{s + tz}$, so $\hat{\mathbf{p}} = \hat{z}\mathbf{r} = \mathbf{p}/(s + tz)$. □

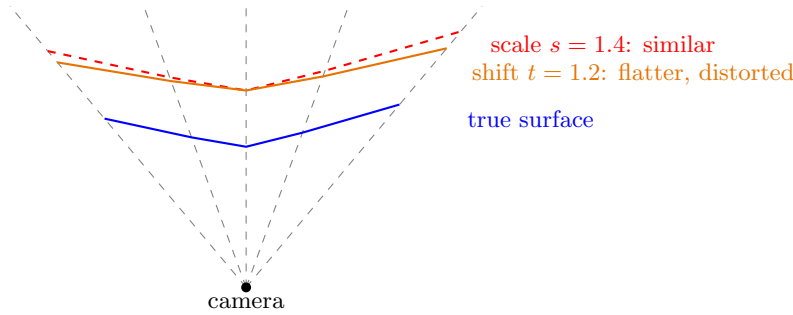


Figure 45: Original side view. A bowl-shaped surface seen along five rays. Scaling by s keeps the shape (red); adding a constant depth t along each ray (orange) keeps the depth differences but widens the lateral spread, so the bowl becomes flatter.

Intuition

Scale is a photocopier enlargement: bigger, same shape. Shift is pushing every point straight away from the camera by the same distance: because rays fan out, far-off directions move sideways more than the centre, so straight things bend. Example: $s = 1$, $t = 1$ m: a point at $z = 2$ is scaled by $1 + 1/2 = 1.5$, one at $z = 10$ by only 1.1; near points are magnified more, so a wall receding from the camera is bent.

3.13.2 Sources of Scale and Shift (*slides 34*)

Two numbers are required, and a single image cannot supply them. Possible sources:

- a **metric-depth model**; camera-aware models narrow the generalization gap by taking the intrinsics as input (3.8);
- a **known baseline** at training time (3.9);
- a **known object size**: the apparent-size cue, which needs recognition;
- **known camera height plus a flat-ground assumption**: the vertical-position cue, which needs flat ground;
- or a **handful of real depth measurements**.

3.14 Marigold-DC (*slides 35–38*)

Section 3.13: guidance from sparse depth measurements. Nothing is retrained: the denoising loop is modified at test time to move the sample toward the available measurements.

3.14.1 The Depth Completion Task (*slides 36*)

Definition 3.14.1 (Depth completion). Depth completion upgrades sparse depth measurements into dense depth maps guided by a conventional image. The sparse points come from the LiDAR returns of Part I, structure-from-motion landmarks, or a phone’s depth sensor.

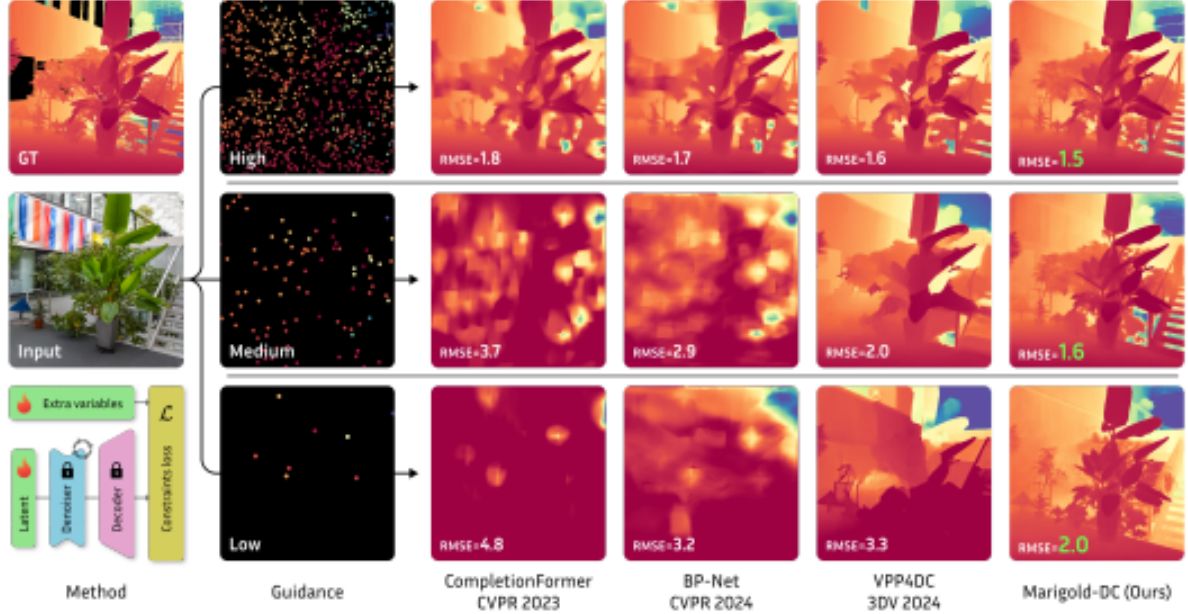


Figure 46: the comparison grid: ground truth, input image and the sparse guidance at high / medium / low density, with the outputs of CompletionFormer, BP-Net, VPP4DC and Marigold-DC and their RMSE labels

guidance density	CompletionFormer	BP-Net	VPP4DC	Marigold-DC
high	1.8	1.7	1.6	1.5
medium	3.7	2.9	2.0	1.6
low	4.8	3.2	3.3	2.0

RMSE values printed on slide 36 (units not stated on the slide).

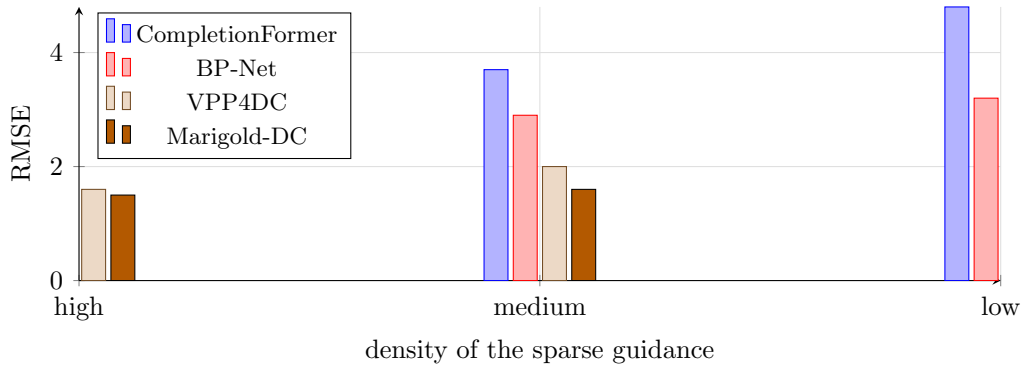


Figure 47: Plot of the RMSE numbers on slide 36. Baselines degrade quickly as the guidance gets sparser; Marigold-DC degrades slowly because the prior fills what the sensor did not sample.

3.14.2 Steering the Loop (*slides 37*)

Depth completion is reframed as *image-conditional depth generation guided by sparse measurements*. It is **generation rather than regression**: the output comes from a prior. The image **conditions** the network, the sparse depth **guides** the loop, and the measurements were never part of training. Every step already walks toward the model’s current estimate; a second pull is added, toward the measurements.

Algorithm 3.14.2 (Guided step). At every step of the loop: (1) read the *preview*, the current guess at the finished depth map, $\hat{z}_0^t = z_t - \sigma_t \epsilon_\theta$ (Tweedie formula); (2) decode it and turn it into meters with the scale and the shift, $\hat{d}_t^{(m)} = \hat{a} \mathcal{D}(\hat{z}_0^t) + \hat{b}$; (3) compare it with the few pixels Ω that have a measurement c , $\mathcal{L} = \|\hat{d}_t^{(m)} - c\|_\Omega$; (4) nudge the latent and the two scalars $(\hat{a}, \hat{b})$ toward the measurements by gradient descent on $\mathcal{L}$; (5) take the ordinary DDIM step.

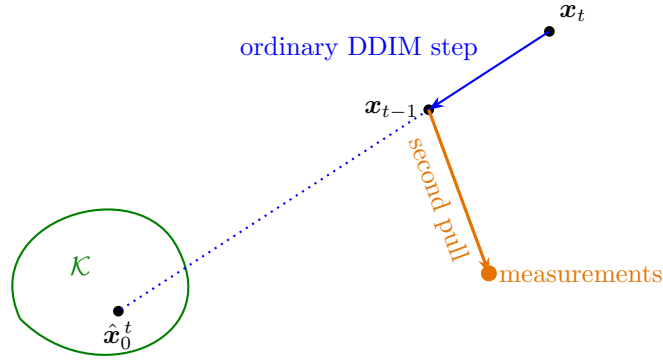


Figure 48: The guided step (redrawn from slide 37): DDIM step plus a second pull toward the sparse measurements.

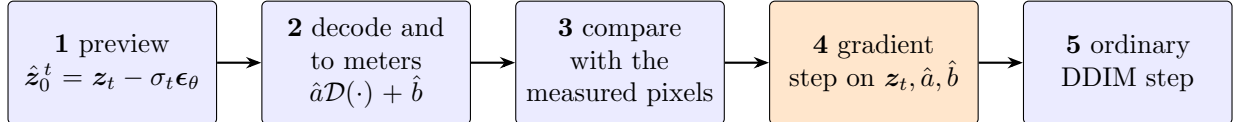


Figure 49: The five steps as a pipeline.

3.14.3 Nothing Retrained (*slides 38*)

Marigold gives affine-invariant depth; the sparse map gives *metric* depth at a few pixels. The unknowns are the **latent**, which decides the shape, and the **scale and shift**, which decide the meters. All three are optimized, for one image, over fifty steps, and then discarded. Gradients are taken with respect to the latent and the two scalars, so **no weights change**: the U-Net and the VAE are frozen throughout.

Theorem 3.14.3 (Zero-shot property, as claimed on the slide). *There is no depth-completion dataset, no architecture change and no new checkpoint. The method is therefore zero-shot across indoor and outdoor scenes, where the baselines ship one checkpoint per domain.*

Intuition

The frozen network is a very good but unit-less sketch artist; the LiDAR points are a few tape-measure readings. At each step the sketch is bent slightly (through the latent) so that it does not contradict the readings, and the ruler (scale and shift) is adjusted until the sketch

reads in meters. Because the readings are used only at test time, the same model handles a room and a street: the debts of Section 1 are paid at once, the prior supplying what the sensor cannot sample and the sensor the two numbers the prior cannot know.

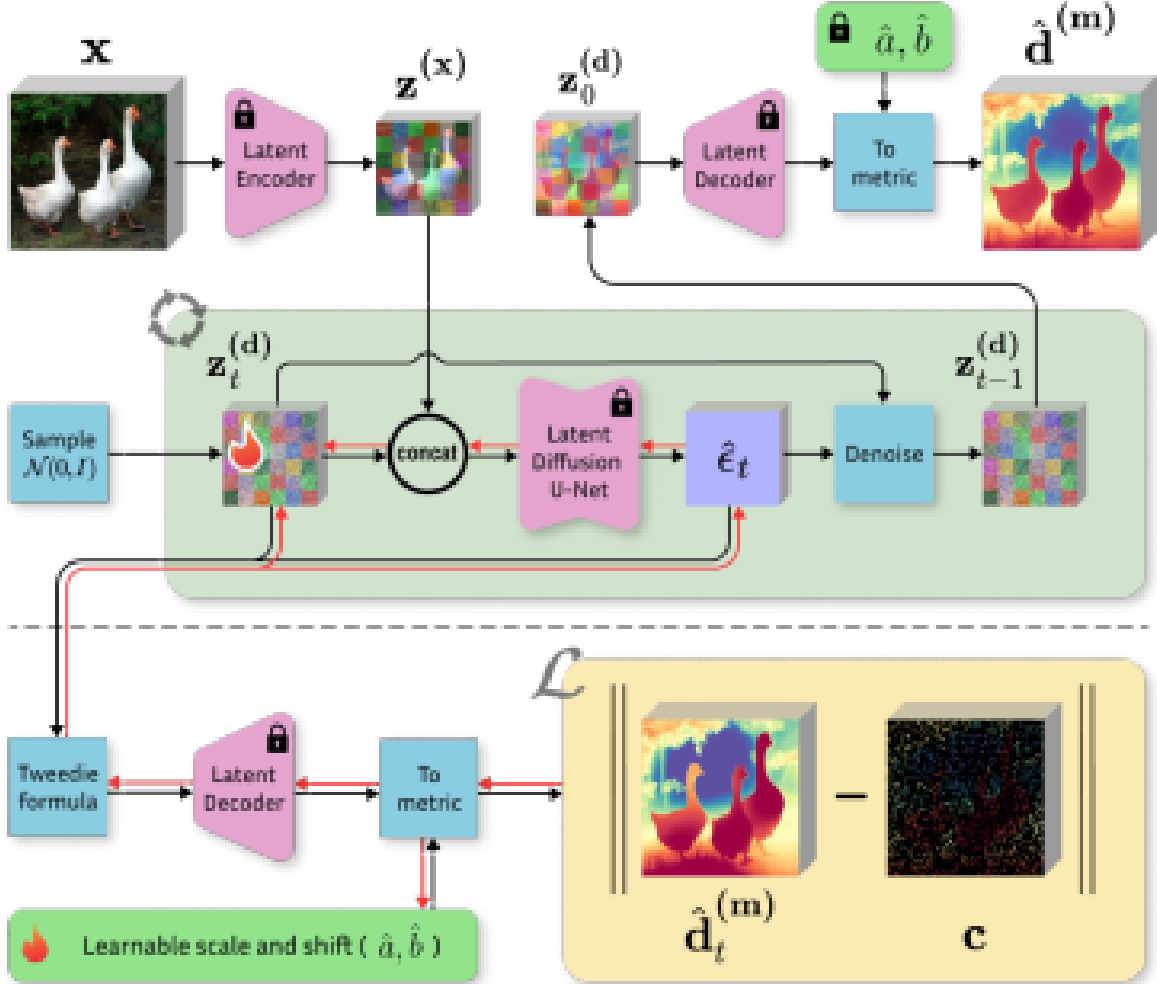


Figure 50: the Marigold-DC diagram: goose photograph, encoder/decoder, frozen U-Net loop with the red gradient arrows, sparse map $\mathbf{c}$ and the resulting metric depth (the block structure is redrawn in the figures above)

Summary (*slides 39–40*)

Comparison of the methods covered

method	pays with	output	strength	main limitation
Pulsed LiDAR	time	metric, sparse	exact where it lands; error nearly range-independent	stripes growing $\propto r$; no return $\neq 0$; mixed points at edges
Rectified stereo	baseline	metric, dense where matched	dense; metric from b	error $\propto z^2/(fb)$; fails on texture-poor, repeated, occluded regions
Structure from motion	computation	shape up to scale	no second camera needed	unknown global scale; rigid scene
Structured light (NYU)	projected pattern	metric, dense indoors	dense labels at short range	inpainted holes; indoor only
Geometric cues (size, vertical position)	assumptions	metric if assumptions hold	closed form, interpretable	need known H , flat ground, known Y and horizon
Eigen scale-invariant loss	prior	depth up to 1 scale	loss respects scale ambiguity	one scale per scene; dataset-specific sensor errors
MiDaS	prior, mixed data	affine- invariant disparity	mixes datasets with no common zero	not metric; shape only
Depth Anything V2	prior, pseudo-labels	affine- invariant inverse depth	scales with unlabeled real images	not metric without fine-tuning; inherits teacher's prior

Every method in this lecture bought depth with a different resource. The last section bought it with two at once.

method	paid with	when	free parameters left
A laser (3.3, 3.4)	time	test	0, but sparse
Stereo	a known baseline	test	0, error grows as z^2
Structure from motion	viewpoints, no baseline	test	1, one global scale
Supervised (3.7)	a prior, on sensor labels	train	0, 1 or 2, set by the labels
Self-supervision (3.9)	a baseline	train	0, for that rig
Marigold (3.11)	a generative prior	train	2
Marigold-DC (3.13)	a prior and time	both	0, dense, nothing retrained

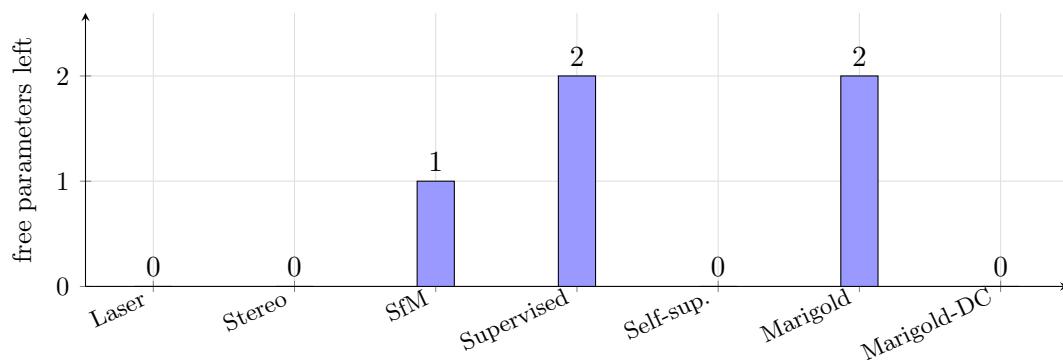


Figure 51: Free parameters left after each method (last column of the table). “Supervised” is drawn at its worst case: it ranges from 0 to 2 depending on the labels.

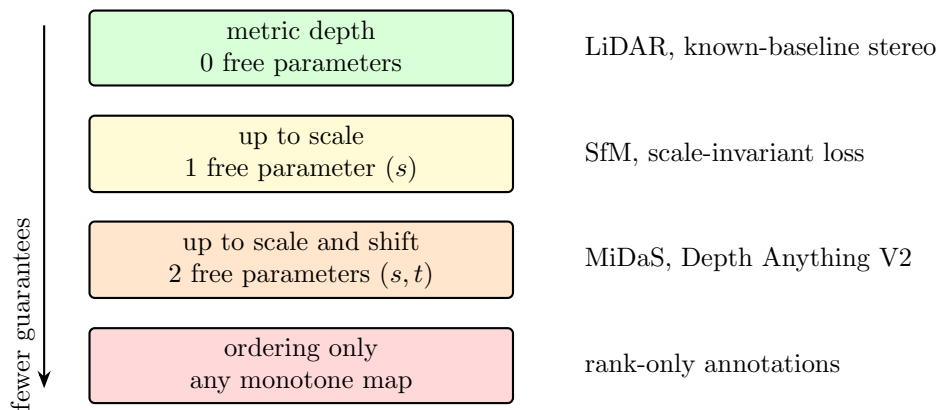


Figure 52: Output spaces ordered by how much of the depth is actually determined. Moving down the ladder, the claim weakens but the data requirement does too.

What each method lets you claim

Key take-aways

- **Depth from one pixel is undetermined.** Back-projection returns a ray; metric depth always needs a *currency*: time (LiDAR), a baseline (stereo) or a prior (learning). A single image bypasses cost, sparsity and hardware, but not the physics.
- **The two physical principles fail differently.** Time of flight has a range-independent timing floor but sparse, structured samples; triangulation is dense but its error grows as $z^2/(fb)$. LiDAR labels must be converted with the camera-frame z , a z-buffer and a validity mask.
- **A learned depth network is a prior, and its output is ambiguous.** The likelihood is flat along each ray, so scale-invariant and affine-invariant losses (in log space or in disparity) ask the network only for shape; disparity is preferred because planes stay planes under an affine map in inverse depth.
- **The prior is both the method and the failure mode.** Networks read vertical image position more than apparent size, only partially respect pitch and roll, and absorb the training focal length; anamorphic art defeats them with no sign of uncertainty.
- **Always ask “recovered or ambiguous?”** Whether a reported depth is in meters, up to scale, or up to scale and shift determines what it can be used for (planning versus shape) and is the key to reading benchmark numbers (aligned first, then scored with AbsRel and δ).
- **The prior supplies what the sensor cannot sample, and the sensor supplies the two numbers the prior cannot know.**
- Self-supervision from stereo needs no depth labels, but the photometric loss is blind on flat and occluded pixels (hand-written priors fill the gap) and is metric only for the baseline it was trained with.
- A diffusion denoiser is a prior: it learns the gradient of a smoothed distance to the data, is defined everywhere, and can be sampled, conditioned, or guided; regression by contrast returns the conditional mean, which blurs depth edges.
- Marigold reuses Stable Diffusion’s weights: frozen VAE, U-Net fine-tuned with the image latent concatenated; percentile normalization makes the output affine-invariant, so it gives shape but no meters, no common zero, and costs $50 \times 10 = 500$ evaluations.

- An affine error $\hat{z} = sz + t$ maps to $\hat{\mathbf{p}} = (s + t/z)\mathbf{p}$: scale is harmless to shape, shift distorts it. Marigold-DC fixes both by optimizing the latent, scale and shift at test time against a few metric points, with no retraining.

Acknowledgments and sources (*slides 41–42*)

The diffusion section (3.10) is adapted from *Introduction to Diffusion*, MIT 6.S183, IAP 2026, Lecture 1, by D. Pfrommer; parts of both halves are adapted from *Depth Estimation* (Politecnico di Milano, 2024) by L. Magri. The reference list on slide 42 (Pfrommer, Yuan, Ke et al., Rombach et al., Yang et al., Ranftl et al., Godard et al., Ho–Jain–Abbeel, Song–Meng–Ermon, Karras et al., and others) and further reading (Yuan’s smallldiffusion) are given there.

A Supplementary derivations (not on the slides)

The following results are *not* on the slides; they are added to justify remarks made in the main text.

Proposition A.1 (DDIM step = re-noising the clean estimate). *Equation (3.11.6) is equivalent to $\mathbf{x}_{t-1} = \hat{\mathbf{x}}_0^t + \sigma_{t-1} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, \sigma_t)$.*

Proof. $\hat{\mathbf{x}}_0^t = \mathbf{x}_t - \sigma_t \boldsymbol{\epsilon}_\theta$, hence $\mathbf{x}_t = \hat{\mathbf{x}}_0^t + \sigma_t \boldsymbol{\epsilon}_\theta$ and $\mathbf{x}_{t-1} = \mathbf{x}_t + (\sigma_{t-1} - \sigma_t) \boldsymbol{\epsilon}_\theta = \hat{\mathbf{x}}_0^t + \sigma_{t-1} \boldsymbol{\epsilon}_\theta$. $\square$

Reading: keep the current clean estimate and re-attach the *same* predicted noise, only with the smaller amplitude σ_{t-1} .

Proposition A.2 (Irreducible floor of the training loss). *For the ideal denoiser, $L_{\text{simple}}(\boldsymbol{\epsilon}^*) = \mathbb{E}_{\mathbf{x}_\sigma, \sigma} \text{tr Cov}(\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma)$, which is strictly positive whenever the posterior of $\mathbf{x}_0$ given $\mathbf{x}_\sigma$ is not a point mass.*

Proof. Applying the conditional-mean decomposition of Section 3.1 with $Y = \boldsymbol{\epsilon}$ and $f = \mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma]$ gives $\mathbb{E}\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}^*\|^2 = \mathbb{E} \text{tr Cov}(\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma)$; and $\boldsymbol{\epsilon} = (\mathbf{x}_\sigma - \mathbf{x}_0)/\sigma$, so $\text{Cov}(\boldsymbol{\epsilon} \mid \mathbf{x}_\sigma) = \text{Cov}(\mathbf{x}_0 \mid \mathbf{x}_\sigma)/\sigma^2$. $\square$

Reading: this explains why the slide-14 curve stops around 0.41 rather than 0: the remaining loss is genuine ambiguity about which data point the noisy input came from, not an optimization failure.